

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

Дисциплина: Технологии проектирования программного обеспечения

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему

**ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ
ТРЕНАЖЕРНОГО ЗАЛА**

БГУИР КП 6-05-0612-03 018 ПЗ

Студент гр. 420602

В. А. Галкин

Руководитель:

Т. С. Боброва

Минск 2026

РЕФЕРАТ

ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ
ТРЕНАЖЕРНОГО ЗАЛА: курсовой проект / В. А. Галкин – Минск : БГУИР,
2026, – п.з. – 50 с., чертежей (плакатов) – 2 л. формата А1.

Предметом проектирования является информационная система тренажерного зала, предназначенная для автоматизации управления клиентами, абонементами, расписанием тренировок, индивидуальными записями, платежами, уведомлениями и отчетностью администрации фитнес-организации.

Целью выполнения курсового проекта является разработка объектно-ориентированной модели информационной системы, обеспечивающей удобное взаимодействие клиента, тренера и администратора, а также позволяющей сократить количество ручных операций при записи на тренировку и контроле активности абонементов.

В ходе выполнения курсового проекта проведен анализ предметной области, сформулированы требования к системе, составлен глоссарий, выполнено моделирование бизнес-процессов, подготовлены описания вариантов использования, разработаны *UML*-диаграммы, а также определена структура базы данных и программных компонентов системы.

В качестве основных методов использованы объектно-ориентированный анализ и проектирование, *UML*-моделирование, описание требований в форме функциональных и нефункциональных спецификаций, а также декомпозиция предметной области на сущности, контроллеры и внешние сервисы.

Результатом проектирования является целостная модель информационной системы тренажерного зала. Полученная модель может быть использована как основа для разработки программного приложения, включающего мобильный интерфейс клиента, личный кабинет тренера, административную панель, сервер приложений, базу данных, а также интеграцию с сервисами уведомлений и оплаты.

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

УТВЕРЖДАЮ
Заведующий кафедрой

(подпись)

2026 г.

ЗАДАНИЕ
по курсовому проектированию

Студенту Галкину Владиславу Александровичу

1. Тема проекта Проектирование информационной системы тренажерного зала

2. Срок сдачи студентом законченного проекта 1 июня 2026 г.

3. Исходные данные к проекту: Информационная система тренажерного зала предназначена для автоматизации учета клиентов, тренеров, абонементов, расписания занятий, записей на тренировки, платежей и уведомлений.

В базе данных системы должны храниться сведения о клиентах, тренерах, администраторах, тарифах, абонементах, платежах, слотах расписания, заявках на тренировку и текущих статусах записей.

Клиенту предоставляется возможность регистрации и входа в систему, просмотра профиля, доступного расписания и статуса абонемента, покупки или продления абонемента, записи на тренировку, отмены записи и получения уведомлений о результате обработки заявки.

Тренер работает с собственным расписанием, создает и изменяет тренировочные слоты, просматривает заявки клиентов, подтверждает или отклоняет записи, а также отмечает факт проведения тренировки.

Администратор управляет клиентами, тренерами, тарифами, абонементами, расписанием, платежами и проблемными заявками, а также формирует отчетную информацию о работе тренажерного зала.

Система должна обеспечивать проверку активности абонемента перед записью, контроль доступности выбранного слота, разграничение прав

доступа, целостность данных и взаимодействие с внешними сервисами оплаты и уведомлений.

4. Содержание расчетно-пояснительной записки (перечень вопросов, которые подлежат разработке)

Введение

1. Анализ технического задания на курсовое проектирование системы

2. Проектирование информационной системы тренажерного зала

3. Программная реализация системы

Заключение

Список использованных источников

Приложения

Ведомость курсового проекта

5. Перечень графического материала (с точным обозначением обязательных чертежей и графиков)

1. Диаграмма вариантов использования (ПЛ, формат А1)

2. Диаграмма классов (ПЛ, формат А1)

6. Консультант по проекту (с обозначением разделов проекта)

Т. С. Боброва

7. Дата выдачи задания – 13 февраля 2026 г.

8. Календарный график работы над проектом на весь период проектирования (с обозначением сроков выполнения и трудоемкости отдельных этапов):

раздел 1 к 15.03 – 30%

раздел 2 к 15.04 – 60%

раздел 3 к 15.05 – 90%

оформление пояснительной записки и графического материала к 20.05 – 100%

Защита курсового проекта с 20.05.2026 по 05.06.2026

РУКОВОДИТЕЛЬ

Т. С. Боброва

(подпись)

Задание принял к исполнению

В. А. Галкин

(дата и подпись студента)

СОДЕРЖАНИЕ

Введение	4
1 Анализ технического задания на курсовое проектирование	7
1.1 Постановка задачи на проектирование	7
1.2 Составление глоссария проекта	9
1.3 Описание дополнительных спецификаций к проекту	12
2 Проектирование системы	15
2.1 Моделирование бизнес-процессов в <i>BPMN</i>	15
2.2 Создание диаграммы вариантов использования	15
2.3 Описание основных вариантов использования	23
2.4 Создание диаграмм взаимодействия	30
2.5 Создание диаграммы классов	33
2.6 Создание диаграммы состояний	34
2.7 Создание диаграммы активности	37
3 Программная реализация системы	39
3.1 Создание диаграммы компонентов	39
3.2 Создание диаграммы развертывания	40
3.3 Генерация кода	42
Заключение	42
Список использованных источников	43
Приложение А (обязательное) Программный код	45

ВВЕДЕНИЕ

В современном мире эффективность работы организаций сферы услуг во многом зависит от скорости обработки информации, удобства взаимодействия с клиентами и точности учета внутренних процессов. Все больше предприятий переходят от ручного ведения данных к использованию информационных систем, позволяющих автоматизировать повторяющиеся операции, снизить количество ошибок и обеспечить быстрый доступ к необходимой информации. Особенно важна автоматизация для организаций, где ежедневно выполняется запись клиентов, ведется расписание, контролируются оплаты и обрабатываются заявки.

К таким организациям относятся тренажерные залы и фитнес-центры. Их деятельность связана с постоянной обработкой информации о клиентах, тренерах, абонементных, расписании тренировок, записях на занятия и платежах. Клиенту необходимо быстро получать сведения о доступных тренировках, проверять состояние абонемента, записываться на занятие или отменять запись. Тренеру важно видеть собственное расписание, управлять тренировочными слотами и своевременно обрабатывать заявки клиентов. Администратор отвечает за ведение клиентской базы, учет тренеров, контроль абонементов, платежей, расписания и спорных ситуаций.

При отсутствии единой информационной системы данные могут храниться разрозненно: в таблицах, журналах, мессенджерах или отдельных файлах. Такой подход усложняет контроль работы тренажерного зала и повышает вероятность ошибок. Например, клиент может быть записан на уже занятое время, заявка может быть обработана с задержкой, абонемент может оказаться недействительным только в момент посещения, а администратору потребуется дополнительное время для проверки платежей и расписания. Поэтому проектирование информационной системы тренажерного зала является актуальной задачей.

Проектируемая информационная система предназначена для автоматизации учета клиентов, тренеров, администраторов, тарифов, абонементов, платежей, тренировочных слотов, заявок на занятия и статусов записей. В системе должны храниться данные о пользователях, их ролях, контактной информации, действующих абонементных, истории оплат, расписании тренировок и результатах обработки заявок. Использование единой базы данных позволит исключить дублирование информации, ускорить поиск нужных сведений и обеспечить более точный контроль процессов тренажерного зала.

Основными пользователями системы являются клиент, тренер и администратор. Клиент после регистрации и входа в систему получает доступ к личному кабинету, где может просматривать данные профиля, актуальное расписание, статус абонемент, доступные тренировки и историю своих записей. Также клиент может приобрести или продлить абонемент, оформить заявку на тренировку, отменить запись и получить уведомление о подтверждении или отклонении заявки.

Тренер в проектируемой системе работает с собственным расписанием. Он может создавать и изменять тренировочные слоты, просматривать список заявок от клиентов, подтверждать или отклонять запись, а также отмечать факт проведения тренировки. Это позволяет тренеру своевременно контролировать занятость, видеть количество записавшихся клиентов и избегать пересечений в расписании.

Администратор выполняет функции управления основными данными системы. Он добавляет и редактирует сведения о клиентах и тренерах, управляет тарифами и абонементом, контролирует платежи, просматривает заявки, решает проблемные ситуации и формирует отчетную информацию о работе тренажерного зала. Благодаря этому администратор получает единый инструмент для контроля деятельности организации.

Важной частью проектируемой системы является автоматическая проверка условий перед записью клиента на тренировку. Система должна определять, активен ли абонемент клиента, доступен ли выбранный тренировочный слот, не превышено ли допустимое количество участников и имеет ли пользователь право выполнять выбранное действие. Такие проверки позволяют снизить количество ошибок, исключить запись без действующего абонемента и предотвратить конфликты в расписании.

Проектирование информационной системы тренажерного зала выполняется с применением объектно-ориентированного подхода и средств визуального моделирования. Для описания структуры и поведения системы используются диаграммы *UML*, позволяющие отразить варианты использования, взаимодействие объектов, структуру классов, состояния записей и последовательность выполнения основных процессов [1]. Бизнес-процессы предметной области моделируются с использованием нотации *BPMN*, что позволяет наглядно представить существующий и проектируемый порядок записи клиента на тренировку, обработки заявок и взаимодействия между клиентом, тренером и администратором. Для построения *BPMN*-диаграмм и части *UML*-диаграмм использовался *Draw.io*. Для компонентной модели и генерации скелетного кода применялось *CASE*-средство *Rational Rose* [2].

Целью курсового проекта является проектирование информационной системы тренажерного зала, обеспечивающей автоматизацию учета клиентов, тренеров, абонементов, расписания, записей на тренировки, платежей и уведомлений.

Для достижения поставленной цели необходимо решить следующие задачи: проанализировать предметную область тренажерного зала; определить основные роли пользователей системы; сформировать требования к функциональности; описать ключевые варианты использования; разработать объектно-ориентированную модель системы; определить основные сущности базы данных; спроектировать взаимодействие между клиентом, тренером, администратором и системой; подготовить модель, пригодную для дальнейшей программной реализации.

Объектом проектирования является деятельность тренажерного зала, связанная с обслуживанием клиентов, организацией тренировочного процесса и административным учетом. Предметом проектирования является информационная система, автоматизирующая запись на тренировки, учет абонементов, управление расписанием, обработку платежей и взаимодействие пользователей через уведомления.

Результатом выполнения курсового проекта является комплексная проектная модель информационной системы тренажерного зала, включающая описание требований, глоссарий предметной области, модели бизнес-процессов, диаграмму вариантов использования, описания основных вариантов использования, диаграммы взаимодействия, классов, состояний, активности, структуру базы данных, а также диаграммы компонентов и размещения. Полученная модель может быть использована как основа для дальнейшей программной реализации приложения, которое позволит клиентам самостоятельно записываться на тренировки, тренерам – управлять своим расписанием и обрабатывать заявки, а администраторам – контролировать клиентов, абонементы, платежи и работу тренажерного зала в единой системе.

1 АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ

1.1 Постановка задачи на проектирование

Предметной областью курсового проекта является деятельность тренажерного зала, связанная с обслуживанием клиентов, организацией тренировочного процесса, управлением расписанием, учетом абонементов и контролем платежей. Работа тренажерного зала включает большое количество повторяющихся операций, которые требуют точного хранения данных и своевременной обработки заявок. К таким операциям относятся регистрация клиентов, ведение информации о тренерах, формирование расписания тренировок, запись клиентов на занятия, подтверждение или отклонение заявок, продление абонементов, обработка оплат и формирование отчетной информации.

В обычной работе тренажерного зала участвуют три основные группы пользователей: клиенты, тренеры и администраторы. Каждый из них выполняет собственный набор действий и взаимодействует с системой с разными правами доступа. Клиент использует систему для просмотра расписания, проверки статуса абонемента, записи на тренировку и получения уведомлений. Тренер работает со своим расписанием, создает тренировочные слоты, просматривает заявки клиентов и принимает решение о подтверждении или отклонении записи. Администратор управляет основными данными системы: клиентами, тренерами, тарифами, абонементом, платежами и проблемными заявками.

Основная проблема предметной области заключается в том, что при ручной обработке информации возрастает вероятность ошибок. Например, клиент может быть записан на недоступное время, тренер может не получить информацию о новой заявке, абонемент может быть просрочен, а администратор может обнаружить проблему только после обращения клиента. Также при отсутствии единой базы данных затрудняется поиск информации о клиентах, проверка платежей, контроль загруженности тренеров и анализ посещаемости. Поэтому информационная система должна объединять данные тренажерного зала в единую структуру и обеспечивать автоматическую проверку основных условий.

Проектируемая информационная система тренажерного зала предназначена для автоматизации следующих процессов:

- учет клиентов тренажерного зала;
- учет тренеров и их специализаций;

- управление тарифами и абонементом;
- формирование и изменение расписания тренировок;
- создание тренировочных слотов;
- запись клиентов на тренировки;
- подтверждение и отклонение заявок тренером;
- проверка активности абонента перед записью;
- проверка доступности выбранного слота;
- обработка платежей;
- отправка уведомлений клиентам;
- формирование отчетной информации для администратора.

В системе должны храниться сведения о клиентах, тренерах, администраторах, тарифах, абонементе, платежах, слотах расписания, заявках на тренировку и текущих статусах записей. Основой работы системы является база данных, в которой сохраняется вся информация, необходимая для корректного выполнения бизнес-процессов тренажерного зала. Использование базы данных позволяет избежать дублирования информации, обеспечить быстрый поиск нужных сведений и сохранить историю действий пользователей.

Клиент начинает работу с системой с регистрации или входа в личный кабинет. После авторизации он получает доступ к данным профиля, информации об активном абонементе и расписанию тренировок. При выборе тренировки система должна проверить, действителен ли абонемент клиента и доступен ли выбранный временной слот. Если абонемент неактивен, клиент получает отказ в записи. Если слот недоступен, система также сообщает о невозможности записи. Если все условия выполнены, создается заявка на тренировку со статусом ожидания подтверждения.

Тренер после входа в систему работает с личным расписанием. Он может создавать новые тренировочные слоты, указывая дату, время начала, продолжительность, тип тренировки и максимальное количество участников. При сохранении слота система должна проверить отсутствие пересечений с уже существующими слотами. Также тренер просматривает заявки клиентов на свои тренировки и принимает решение: подтвердить запись или отклонить ее. После подтверждения запись фиксируется в расписании, а клиент получает уведомление о результате.

Администратор выполняет функции управления системой. Он может просматривать список тренеров, добавлять новых тренеров, изменять их данные или удалять их из системы. При добавлении тренера необходимо указать ФИО, специализацию, контактные данные, email, логин, пароль и при необходимости стаж работы. Система должна проверять корректность

введенных данных, уникальность email и логина. Если тренер связан с активными или будущими записями клиентов, его полное удаление может привести к нарушению целостности данных, поэтому в таком случае целесообразно переводить учетную запись тренера в неактивный или архивный статус.

Важным процессом является запись клиента на тренировку. Данный процесс включает несколько этапов: выбор клиентом слота в расписании, проверку активности абонемента, проверку доступности выбранного слота, создание заявки на запись, направление запроса тренеру, принятие решения тренером и фиксацию результата в системе. Если запись подтверждена, система обновляет статус заявки и уведомляет клиента. Если запись отклонена, клиент получает сообщение об отказе. Такой порядок позволяет контролировать не только техническую доступность слота, но и решение тренера.

Для корректной работы информационной системы необходимо обеспечить разграничение прав доступа. Клиент не должен иметь возможности изменять расписание тренера или просматривать административные данные. Тренер должен иметь доступ только к собственному расписанию и заявкам на свои тренировки. Администратор должен иметь расширенные права для управления пользователями, абонементом, платежами и отчетами. Разграничение ролей повышает безопасность системы и предотвращает случайное или намеренное изменение данных пользователями, не имеющими соответствующих полномочий.

Таким образом, проектируемая информационная система должна обеспечить централизованное хранение данных, автоматизацию записи клиентов на тренировки, контроль активности абонементов, управление расписанием и поддержку взаимодействия между клиентом, тренером и администратором. В результате внедрения такой системы сокращается количество ручных операций, повышается точность учета и улучшается качество обслуживания клиентов тренажерного зала.

1.2 Составление глоссария проекта

Глоссарий проекта предназначен для описания основных терминов предметной области. Он позволяет единообразно использовать понятия в процессе анализа, проектирования и дальнейшей реализации информационной системы. Основные термины проекта представлены в таблице 1.1.

Таблица 1.1 – Глоссарий проекта

Термин	Определение
Информационная система тренажерного зала	Программная система, предназначенная для автоматизации учета клиентов, тренеров, абонементов, расписания, записей на тренировки, платежей и уведомлений.
Пользователь	Обобщенная сущность системы, включающая клиента, тренера и администратора. Пользователь имеет учетную запись, логин, пароль, контактные данные и роль в системе.
Клиент	Пользователь тренажерного зала, который может просматривать расписание, проверять статус абонемента, записываться на тренировки, отменять запись и получать уведомления.
Тренер	Пользователь системы, который проводит тренировки, создает и изменяет тренировочные слоты, просматривает заявки клиентов, подтверждает или отклоняет записи.
Администратор	Пользователь с расширенными правами доступа, управляющий клиентами, тренерами, тарифами, абонементами, расписанием, платежами и отчетной информацией.
Абонемент	Запись, подтверждающая право клиента посещать тренажерный зал или тренировки в течение определенного периода или по выбранному тарифу.
Тариф	Условия предоставления абонемента, включающие тип посещения, стоимость, срок действия и возможные ограничения.
Платеж	Операция оплаты услуг тренажерного зала.

Продолжение таблицы 1.1

Расписание	Совокупность тренировочных слотов, доступных для просмотра клиентами и управления тренерами или администраторами.
Слот расписания	Конкретный временной интервал тренировки, содержащий дату, время начала, продолжительность, тип тренировки, тренера, максимальное количество участников и статус доступности.
Запись на тренировку	Заявка клиента на посещение выбранного тренировочного слота. Может иметь статус ожидания, подтверждения, отклонения, отмены или проведения.
Заявка	Обращение клиента на запись к тренеру, требующее проверки системой и решения тренера.
Статус записи	Текущее состояние записи клиента на тренировку: создана, ожидает подтверждения, подтверждена, отклонена, отменена, проведена или проблемная.
Уведомление	Сообщение, отправляемое клиенту или другому пользователю системы о результате обработки заявки, изменении расписания, оплате или другом событии.
Личный кабинет клиента	Раздел системы, в котором клиент просматривает профиль, абонемент, расписание, записи и уведомления.
Личный кабинет тренера	Раздел системы, в котором тренер управляет расписанием, просматривает заявки клиентов и отмечает проведение тренировок.
Административная панель	Интерфейс администратора для управления пользователями, абонементом, платежами, расписанием и отчетами.

Продолжение таблицы 1.1

Активный абонемент	Абонемент, срок действия которого не истек и который позволяет клиенту записываться на тренировки.
Неактивный абонемент	Абонемент, который не дает клиенту права на запись из-за окончания срока действия, аннулирования или отсутствия оплаты.
Проблемная заявка	Заявка, требующая вмешательства администратора, например при отсутствии ответа тренера, конфликте расписания или ошибке оплаты.
База данных	Хранилище информации о пользователях, абонементах, платежах, расписании, заявках и статусах записей.
Сервис уведомлений	Внешний или внутренний компонент, обеспечивающий отправку уведомлений пользователям по <i>Email</i> , <i>SMS</i> или <i>Push</i> -каналу.
Банковская система	Внешняя система, используемая для проведения и проверки платежей за абонементы или услуги тренажерного зала.

1.3 Описание дополнительных спецификаций к проекту

Функциональные возможности. Информационная система тренажерного зала должна обеспечивать хранение и отображение информации о клиентах, тренерах, администраторах, абонементах, тарифах, расписании занятий, записях на тренировки, платежах и уведомлениях. Система обязана предоставлять клиенту возможность регистрации, входа в личный кабинет, просмотра доступного расписания, проверки статуса абонемента, покупки или продления абонемента, записи на тренировку и отмены записи. Для тренера должны быть реализованы функции просмотра собственного расписания, создания и изменения тренировочных слотов, просмотра заявок клиентов, подтверждения или отклонения записей, а также отметки факта проведения тренировки. Администратор должен иметь возможность управлять данными

клиентов и тренеров, тарифами, абонементом, платежами, расписанием и проблемными заявками. Особое внимание в системе уделяется проверке активности абонента перед записью клиента на тренировку, проверке доступности выбранного слота и автоматическому обновлению статусов записей.

Пользовательский интерфейс. Интерфейс системы должен быть удобным и понятным для всех категорий пользователей: клиента, тренера и администратора. Клиентская часть должна быть адаптирована для мобильных устройств, так как запись на тренировку чаще всего выполняется через смартфон. Интерфейс клиента должен содержать личный кабинет, раздел просмотра абонента, расписание тренировок, форму записи и раздел уведомлений. Интерфейс тренера должен предоставлять доступ к расписанию, списку заявок и инструментам подтверждения или отклонения записей. Для администратора должна быть предусмотрена *web*-панель управления, позволяющая работать с пользователями, абонентами, платежами, расписанием и отчетной информацией. Интерфейс должен поддерживать работу в современных браузерах на операционных системах *Windows* и *Mac OS*, а клиентская часть должна корректно отображаться на мобильных устройствах *iOS* и *Android*.

Надежность. Система должна обеспечивать стабильную работу в течение времени использования тренажерного зала. Необходимо предусмотреть сохранность данных о клиентах, абонентах, платежах и записях на тренировки. Для предотвращения потери информации должны выполняться регулярные резервные копии базы данных. При возникновении сбоя система не должна допускать повреждения или потери уже сохраненных данных. Восстановление работоспособности после сбоя должно занимать не более 10 минут. Ошибки, связанные с недоступностью отдельных сервисов, например сервиса уведомлений или платежной системы, не должны нарушать работу основных функций системы.

Производительность. Система должна обеспечивать одновременную работу клиентов, тренеров и администратора без значительных задержек. Просмотр расписания, статуса абонента и списка записей должен выполняться не более чем за 3 секунды. Операции создания заявки, подтверждения записи, изменения слота расписания и обновления статуса должны выполняться не более чем за 5 секунд. Система должна выдерживать одновременную работу не менее 100 пользователей, что достаточно для функционирования среднего тренажерного зала или фитнес-центра.

Безопасность. Система должна быть построена таким образом, чтобы исключить несанкционированный доступ к персональным данным клиентов,

информации о платежах, абонементов и расписании тренеров. Доступ к функциям системы должен зависеть от роли пользователя. Клиент не должен иметь возможность изменять расписание тренера или просматривать административные данные. Тренер должен иметь доступ только к собственному расписанию и заявкам клиентов на свои тренировки. Администратор должен обладать расширенными правами управления, но его действия должны фиксироваться в системе. Пароли пользователей должны храниться в защищенном виде. Также система должна предотвращать изменение или удаление данных лицами, не имеющими соответствующих прав доступа.

Проектные ограничения. Система должна функционировать на основе реляционной системы управления базами данных. В базе данных должны храниться сведения о пользователях, ролях, абонементов, тарифах, платежах, тренировочных слотах, заявках, уведомлениях и статусах записей. Проектируемая система должна иметь модульную структуру, позволяющую отдельно реализовать клиентское приложение, личный кабинет тренера, административную панель, серверную часть, базу данных, модуль уведомлений и модуль работы с платежами. Такое разделение позволит упростить дальнейшую разработку, сопровождение и расширение системы.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Моделирование бизнес-процессов в *BPMN*

Бизнес-процесс – это совокупность взаимосвязанных действий, направленных на получение определенного результата для пользователя или организации. В рамках информационной системы тренажерного зала к бизнес-процессам относятся действия, связанные с записью клиента в зал, оформлением абонемента, выбором тренировки, взаимодействием с тренером, подтверждением записи и уведомлением клиента о результате.

Для визуализации бизнес-процессов используется нотация *BPMN* (*Business Process Model and Notation*). Данная нотация позволяет отобразить последовательность действий, участников процесса, точки принятия решений, возможные варианты развития событий и обмен сообщениями между участниками. Использование *BPMN* удобно при проектировании информационной системы, так как позволяет наглядно показать, какие операции выполняются вручную, какие могут быть автоматизированы, и какие участники задействованы на каждом этапе.

Преимущества нотации *BPMN* заключаются в следующем:

1 Детализация процесса – на диаграмме можно отразить все основные этапы работы, исполнителей, события, условия и варианты завершения процесса.

2 Наглядность – схема позволяет быстро понять последовательность действий даже пользователям, не участвующим в разработке программного обеспечения.

3 Выявление проблемных мест – *BPMN*-диаграмма помогает определить этапы, на которых возникают задержки, дублирование действий или зависимость от ручной обработки информации.

Для проектируемой информационной системы тренажерного зала были построены две *BPMN*-диаграммы: модель текущего состояния процесса – *AS-IS* и модель целевого состояния процесса – *TO-BE*. Диаграмма *AS-IS* показывает существующий порядок записи клиента в тренажерный зал, при котором значительная часть действий выполняется вручную через администратора. Диаграмма *TO-BE* показывает целевой вариант работы после внедрения информационной системы, где основные операции выполняются автоматически через сайт или мобильное приложение.

Диаграмма бизнес-процесса *AS-IS* отражает текущий порядок записи клиента в тренажерный зал. В процессе участвуют клиент, администратор и тренер. Основная особенность данной модели заключается в том, что

большинство операций выполняется вручную: клиент обращается лично или по телефону, администратор проверяет наличие свободных мест, уточняет условия посещения и фиксирует данные клиента.

Процесс начинается с желания клиента записаться в зал. После обращения администратор проверяет наличие мест и сообщает клиенту доступные условия. Если условия клиента не устраивают, процесс завершается отказом. Если клиент согласен, администратор записывает его данные, принимает оплату и выдает абонемент.

Далее клиент может выбрать самостоятельное посещение или запись к тренеру. Если требуется занятие с тренером, администратор передает запрос тренеру. Тренер проверяет расписание и сообщает, возможна ли запись. При наличии свободного слота запись фиксируется, клиент получает подтверждение. Если свободного времени нет, клиент получает отказ.

Недостатками модели *AS-IS* являются зависимость от администратора, ручное ведение записей, ожидание ответа тренера, риск ошибок в расписании и отсутствие автоматической проверки данных.

Таким образом, модель *AS-IS* показывает, что существующий процесс записи клиента в тренажерный зал является недостаточно эффективным. Основные данные о клиенте, абонементе, оплате и записи на тренировку обрабатываются разными участниками вручную, из-за чего процесс становится зависимым от внимательности администратора и доступности тренера. При большом количестве клиентов это может приводить к задержкам, ошибкам при проверке расписания и повторному вводу одних и тех же данных.

Особенно важной проблемой является отсутствие автоматической проверки активности абонемента и занятости выбранного тренировочного слота. В результате администратор вынужден самостоятельно контролировать, имеет ли клиент право на посещение, свободен ли тренер в выбранное время и не возникнет ли конфликт расписания. Поэтому текущая модель нуждается в преобразовании, при котором часть ручных операций должна быть перенесена в информационную систему.

Кроме того, в модели *AS-IS* отсутствует единый механизм хранения и обновления информации. Данные о клиенте, оплате, абонементе и записи на тренировку могут фиксироваться в разных местах: в бумажных журналах, электронных таблицах, сообщениях администратора или устных договоренностях с тренером. Из-за этого усложняется контроль актуальности данных и повышается вероятность возникновения противоречий. Например, тренер может считать слот свободным, в то время как администратор уже записал на это время другого клиента.

Еще одним недостатком текущего процесса является слабая прозрачность для клиента. Клиент не всегда может самостоятельно увидеть доступное расписание, проверить статус своего абонемента или узнать, подтверждена ли запись на тренировку. Для получения этой информации ему необходимо обращаться к администратору, что увеличивает время обслуживания и создает дополнительную нагрузку на персонал тренажерного зала.

На рисунке 2.1 представлена диаграмма бизнес-процесса *AS-IS*, отражающая существующий порядок записи клиента в тренажерный зал.

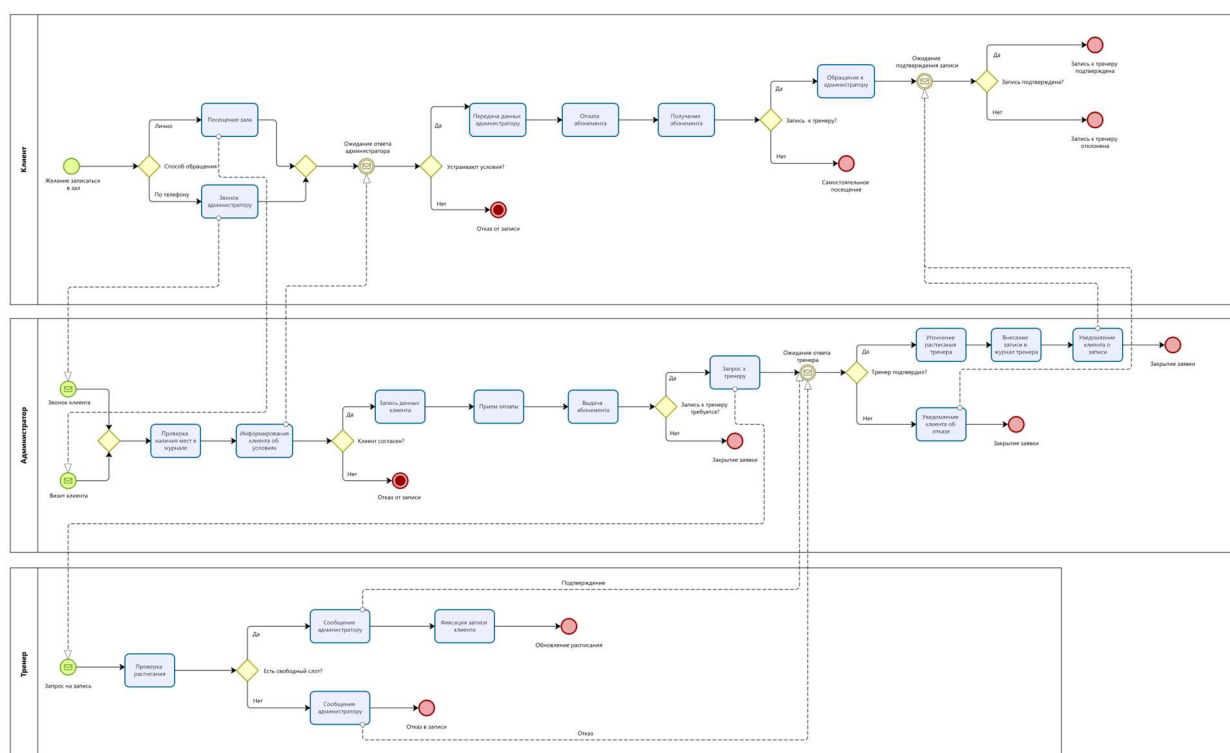


Рисунок 2.1 – Диаграмма бизнес-процесса *AS-IS*

Диаграмма бизнес-процесса *TO-BE* отражает целевой порядок работы после внедрения информационной системы тренажерного зала. В процессе участвуют клиент, информационная система, тренер и администратор. Основная часть операций переносится в систему, что позволяет сократить ручную обработку заявок и ускорить запись клиента.

Процесс начинается с выбора клиентом способа записи через сайт или мобильное приложение. Клиент выбирает тариф и услугу, заполняет данные профиля и отправляет заявку. Информационная система проверяет данные, формирует условия обслуживания и передает их клиенту. Если клиент согласен с условиями, он оплачивает абонемент, после чего система создает электронный абонемент и сохраняет данные клиента в базе.

После получения абонеента клиент выбирает, требуется ли ему запись к тренеру. Если тренер не нужен, клиент переходит к самостоятельному посещению. Если требуется занятие с тренером, клиент выбирает тренера и время, а система отправляет запрос тренеру. Тренер проверяет расписание и подтверждает либо отклоняет запись.

Если запись подтверждена, система фиксирует ее в расписании и отправляет клиенту уведомление. Если тренер отклоняет заявку, клиент получает уведомление об отказе. При отсутствии ответа тренера заявка передается администратору для ручной обработки.

В целевой модели *TO-BE* информационная система становится центральным участником процесса. Она принимает заявку клиента, проверяет введенные данные, обращается к базе данных, контролирует наличие активного абонеента, проверяет доступность выбранного слота и фиксирует результат обработки заявки. За счет этого уменьшается нагрузка на администратора и сокращается время ожидания клиента.

В отличие от модели *AS-IS*, где администратор вручную уточняет условия посещения и связывается с тренером, в модели *TO-BE* большая часть проверок выполняется автоматически. Тренер участвует только на этапе подтверждения или отклонения записи, а администратор подключается в исключительных случаях, например при проблемной заявке, ошибке оплаты или отсутствии ответа тренера. Это делает процесс более устойчивым, прозрачным и удобным для всех участников.

Модель *TO-BE* позволяет автоматизировать регистрацию клиента, оформление абонеента, проверку условий записи, взаимодействие с тренером и отправку уведомлений. Все сведения о клиентах, тренерах, платежах, абонеентах, расписании и статусах записей хранятся в единой базе данных, что обеспечивает целостность информации и упрощает дальнейшее формирование отчетов.

Следовательно, внедрение информационной системы позволяет не только ускорить оформление абонеента и запись на тренировку, но и повысить достоверность данных. Переход от модели *AS-IS* к модели *TO-BE* снижает количество ручных операций, уменьшает вероятность ошибок при проверке расписания и повышает качество обслуживания клиентов тренажерного зала.

На рисунке 2.2 представлена диаграмма бизнес-процесса *TO-BE*, отражающая целевой порядок работы тренажерного зала после внедрения информационной системы.

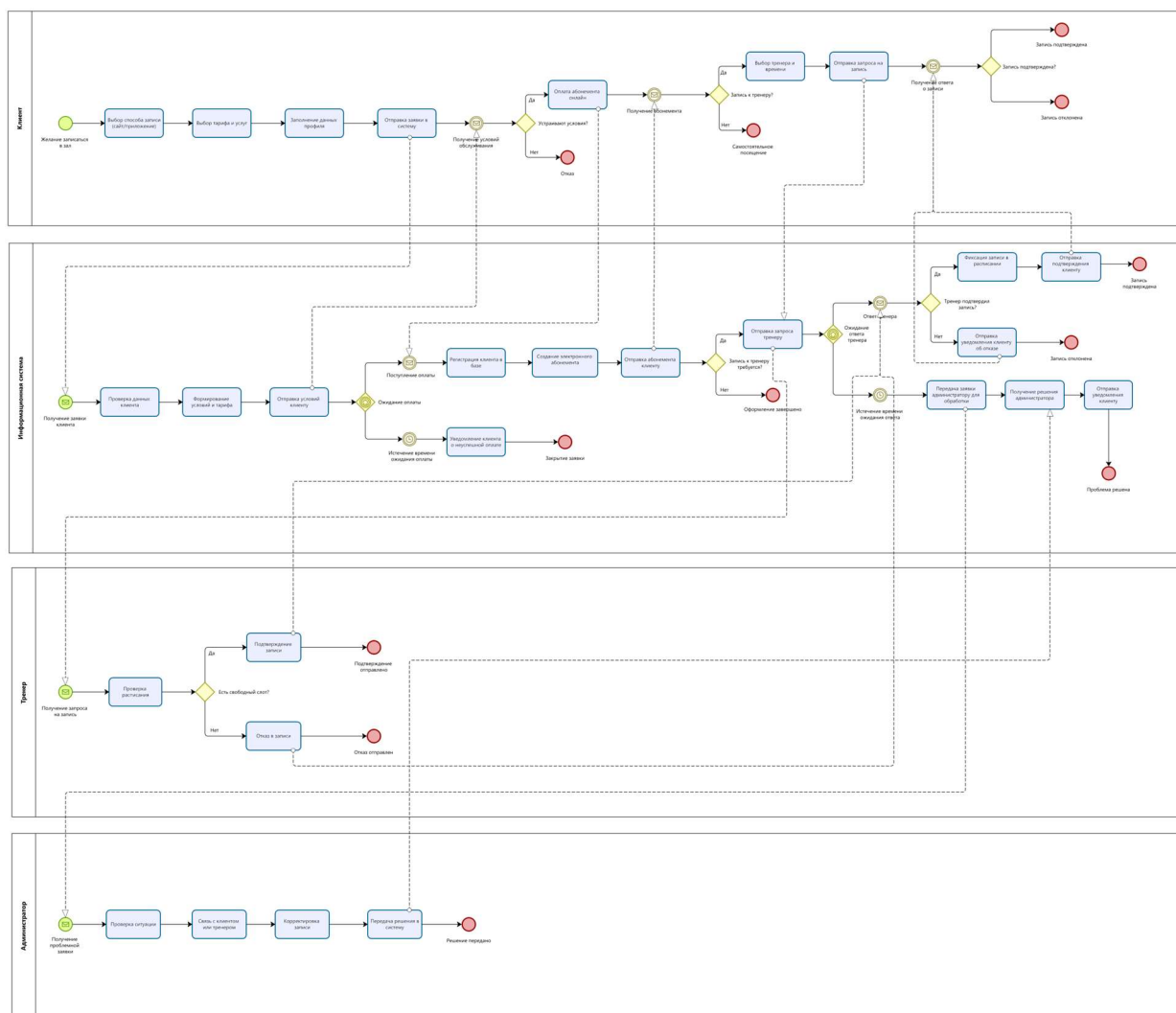


Рисунок 2.2 – Диаграмма бизнес-процесса *TO-BE*

2.2 Создание диаграммы вариантов использования

Use Case-диаграмма является одним из основных инструментов объектно-ориентированного анализа и проектирования. Она предназначена для описания функциональных требований к системе и отображает взаимодействие между внешними участниками процесса и проектируемой информационной системой. С помощью данной диаграммы можно определить границы системы, выделить основные роли пользователей и показать, какие функции доступны каждой категории участников.

Вариант использования представляет собой последовательность действий, выполняемых системой в ответ на запрос действующего лица. Действующее лицо – это пользователь или внешняя система, взаимодействующая с проектируемой системой для достижения определенной цели. В рамках проектирования информационной системы тренажерного зала

варианты использования позволяют описать основные сценарии работы: регистрацию пользователя, покупку и продление абонемента, просмотр расписания, запись на тренировку, управление тренерами, обработку заявок и работу с тренировочными слотами.

Для информационной системы тренажерного зала основными действующими лицами являются клиент, тренер и администратор.

Клиент – это пользователь системы, который получает доступ к личному кабинету и выполняет действия, связанные с получением услуг тренажерного зала. Клиент может зарегистрироваться в системе, просматривать и редактировать профиль, проверять статус абонемента, покупать или продлевать абонемент, просматривать расписание, записываться на тренировку, просматривать свои записи и отменять запись при необходимости. Основная цель клиента при работе с системой заключается в самостоятельном получении информации о доступных услугах и оформлении записи без обязательного обращения к администратору.

Тренер – это пользователь системы, отвечающий за организацию и проведение тренировок. В его функции входит создание слота расписания, просмотр собственного расписания, подтверждение или отклонение записей клиентов, обновление расписания, удаление слота расписания и просмотр списка записей клиентов. Тренер взаимодействует с системой для контроля своей занятости, обработки заявок и своевременного информирования клиентов о результате записи.

Администратор – это пользователь с расширенными правами доступа. Он обеспечивает управление основными данными информационной системы тренажерного зала. Администратор может просматривать список тренеров, добавлять тренеров, изменять данные тренера, удалять тренера, регистрировать клиентов, изменять данные клиента, удалять клиента, продавать абонемент, аннулировать абонемент, просматривать списки клиентов и абонементов, а также обрабатывать проблемные заявки. Роль администратора является важной, так как он контролирует корректность данных, решает спорные ситуации и поддерживает актуальность информации в системе.

На диаграмме вариантов использования отображены основные функции, доступные каждому действующему лицу. Для клиента выделены варианты использования, связанные с личным профилем, абонементом и записью на тренировку. Для тренера показаны действия, связанные с расписанием и обработкой записей клиентов. Для администратора представлены функции управления пользователями, тренерами, абонементом и проблемными заявками.

Особое место на диаграмме занимает вариант использования «Записаться на тренировку», так как он связан с несколькими важными действиями системы. Перед выполнением записи необходимо учитывать наличие действующего абонемента, доступность выбранного слота расписания и возможность подтверждения записи тренером. После обработки заявки клиент получает уведомление о результате. Таким образом, данный вариант использования отражает один из ключевых бизнес-процессов тренажерного зала.

Вариант использования «Покупка абонемента» связан с возможностью дальнейшей записи клиента на тренировку. Если у клиента отсутствует активный абонемент, система должна ограничить возможность записи и предложить оформить или продлить абонемент. Это позволяет обеспечить контроль посещений и исключить запись клиентов без действующего права на получение услуги.

Варианты использования, связанные с тренером, направлены на управление расписанием и обработку заявок клиентов. Тренер может создавать новые тренировочные слоты, просматривать свое расписание, подтверждать или отклонять записи, обновлять расписание и удалять слоты. Это позволяет поддерживать актуальное расписание тренировок и предотвращать конфликты по времени.

Варианты использования администратора направлены на сопровождение работы всей системы. Администратор управляет данными тренеров и клиентов, контролирует абонементы, выполняет регистрацию клиентов, продажу и аннулирование абонементов, а также обрабатывает проблемные заявки. При возникновении нестандартной ситуации администратор может проверить данные в системе, уточнить информацию у клиента и принять решение по дальнейшей обработке заявки.

Также на диаграмме показано, что отдельные варианты использования связаны между собой. Например, запись на тренировку зависит от состояния абонемента клиента и доступности выбранного времени в расписании. Если абонемент неактивен или слот уже занят, система должна ограничить возможность записи и не допустить возникновения ошибки в расписании.

Отдельное значение имеет обработка проблемных заявок. Такие ситуации могут возникать при отмене записи, спорном статусе абонемента, изменении расписания тренера или необходимости связи с клиентом. Наличие данного варианта использования позволяет сохранить автоматизацию основных процессов, но при этом оставить возможность ручного вмешательства администратора в случаях, когда система не может завершить обработку заявки автоматически.

Диаграмма вариантов использования проектируемой информационной системы тренажерного зала представлена на рисунке 2.3.

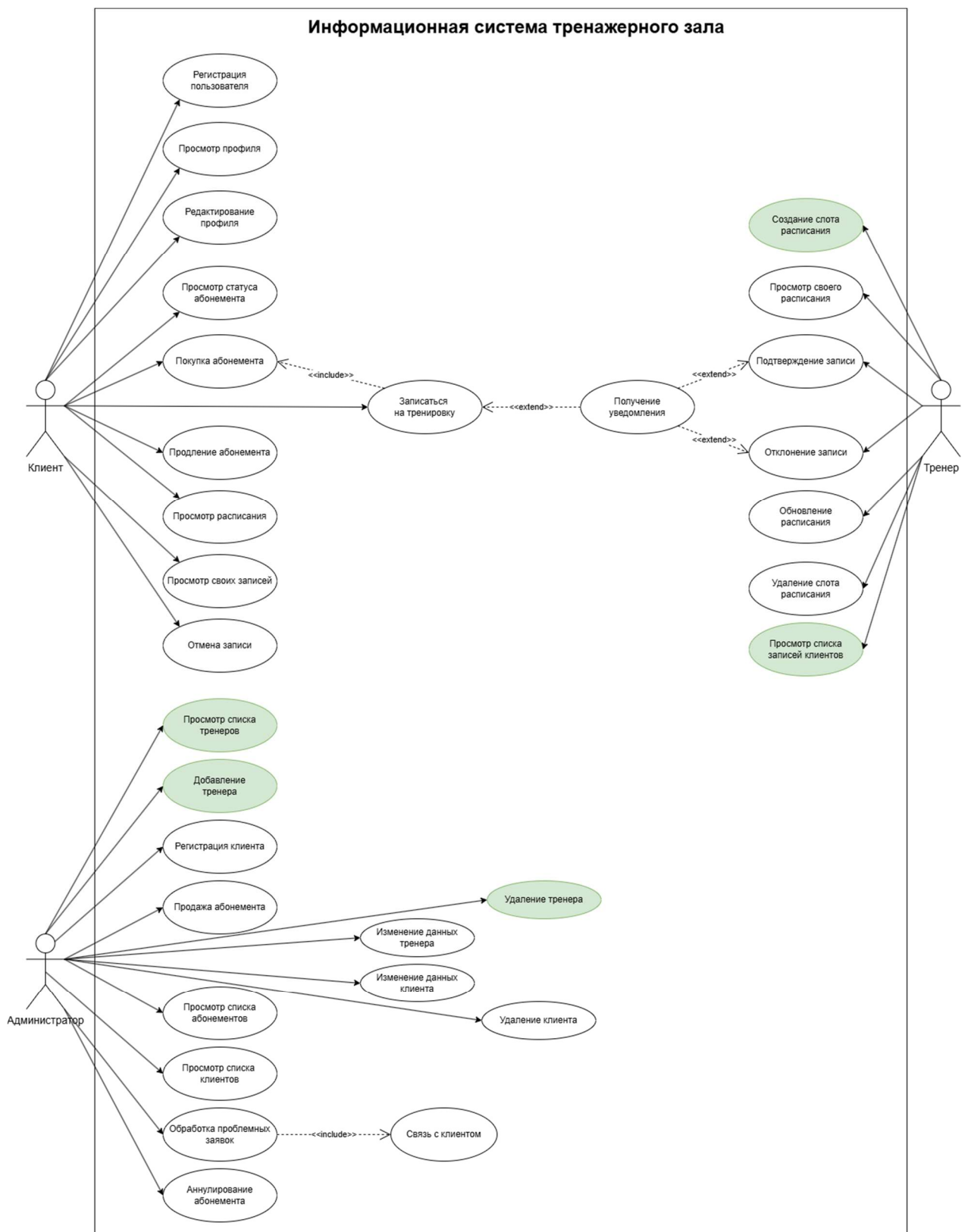


Рисунок 2.3 – Диаграмма вариантов использования

Диаграмма вариантов использования является основой для дальнейшего описания функционала информационной системы тренажерного зала. На ее основе формируются текстовые спецификации, в которых уточняются действия пользователей, предварительные условия, результаты выполнения сценариев, альтернативные направления и исключительные ситуации.

Построенная диаграмма показывает распределение функций между клиентом, тренером и администратором. Она позволяет определить ключевые сценарии работы системы и служит основой для построения диаграмм взаимодействия, классов, состояний и активности.

2.3 Описание основных вариантов использования

После построения диаграммы вариантов использования необходимо выполнить текстовое описание наиболее важных сценариев работы проектируемой информационной системы. Такое описание позволяет уточнить действия пользователей, предварительные условия, результаты выполнения сценария, а также возможные альтернативные и исключительные ситуации. В дальнейшем данные спецификации используются как основа для построения диаграмм взаимодействия, диаграммы классов и логики программной реализации системы.

Для информационной системы тренажерного зала основными действующими лицами являются клиент, тренер и администратор. Клиент использует систему для просмотра расписания, проверки абонемента и записи на тренировку. Тренер работает с собственным расписанием, создает тренировочные слоты и обрабатывает заявки клиентов. Администратор выполняет управление пользователями, тренерами, абонементами, платежами и расписанием.

При описании вариантов использования важно учитывать *CRUDL*-анализ основных сущностей системы. Данный подход предполагает определение базовых операций, выполняемых над объектами информационной системы: создание (*Create*), чтение или просмотр записи (*Read*), изменение (*Update*), удаление (*Delete*) и просмотр списка записей (*List*). Это позволяет установить, какие действия должны быть доступны пользователям, и помогает не упустить важные сценарии при описании вариантов использования. Кроме того, *CRUDL*-анализ упрощает проверку полноты функциональных требований, так как для каждой значимой сущности можно определить необходимый набор операций [5].

В данном подразделе описаны основные варианты использования, связанные с управлением тренерами и расписанием тренировок. Эти сценарии

являются важными для проектируемой системы, так как от корректного ведения данных о тренерах и слотах расписания зависит возможность записи клиентов на занятия и отсутствие конфликтов во времени.

Вариант использования «Просмотр списка тренеров» представлен в таблице 2.1. Основным действующим лицом является администратор.

Таблица 2.1 – Описание варианта использования «Просмотр списка тренеров»

Идентификатор и название	UC-1 Просмотр списка тренеров	
Автор	Владислав	Дата создания: 17.03.2026
Основное действующее лицо	Администратор	
Описание	Администратор запрашивает и просматривает список всех зарегистрированных тренеров для управления персоналом тренажерного зала.	
Триггер	Администратор указывает, что хочет просмотреть перечень тренеров.	
Предварительные условия	<i>PRE-1.</i> Личность действующего администратора аутентифицирована. <i>PRE-2.</i> База данных доступна для чтения.	
Выходные условия	<i>POST-1.</i> Список тренеров отображен на экране администратора. <i>POST-2.</i> Администратор может перейти к просмотру, изменению или удалению выбранной записи тренера.	
Нормальное направление развития варианта использования	Просмотр полного списка тренеров. 1. Администратор переходит в раздел управления тренерами. 2. Система отправляет запрос к базе данных. 3. Система извлекает сведения о зарегистрированных тренерах. 4. Система отображает таблицу со списком тренеров (ФИО тренера, специализация, контактные данные и текущий статус).	

Продолжение таблицы 2.1

Альтернативное направление развития варианта использования	Поиск или фильтрация тренеров. 1. Администратор вводит имя тренера в строку поиска или выбирает фильтр по специализации. 2. Система выполняет поиск по заданным параметрам. 3. Система отображает только тех тренеров, которые соответствуют заданным критериям.
Исключения	Список тренеров пуст. 1. Система определяет, что в базе данных отсутствуют записи о тренерах. 2. Система отображает сообщение: «Список тренеров пуст». По заданным параметрам ничего не найдено. 1. Система отображает сообщение: «По вашему запросу ничего не найдено». 2. Администратор может сбросить фильтр или выйти из раздела.

Таблица 2.2 – Описание варианта использования «Добавление тренера»

Идентификатор	UC-2 Добавление тренера	
Автор	Владислав	Дата создания: 17.03.2026
Основное действующее лицо	Администратор	
Описание	Администратор вносит нового тренера в систему для последующего формирования расписания, ведения записей клиентов и отображения информации о тренере.	
Триггер	Администратор указывает, что хочет добавить нового тренера в систему.	
Предварительные условия	<i>PRE-1.</i> Личность действующего администратора аутентифицирована. <i>PRE-2.</i> База данных доступна для записи.	
Выходные условия	<i>POST-1.</i> Новый тренер сохраняется в базе данных. <i>POST-2.</i> Тренер отображается в списке тренеров. <i>POST-3.</i> Тренер становится доступен для	

Продолжение таблицы 2.2

	формирования расписания и записи клиентов на тренировки.
Нормальное направление развития варианта использования	Добавление нового тренера. 1. Администратор открывает раздел управления тренерами. 2. Администратор выбирает действие «Добавить тренера». 3. Система отображает форму добавления нового тренера. 4. Администратор вводит данные тренера: ФИО, специализацию, номер телефона, адрес электронной почты, логин, пароль и стаж работы. 5. Администратор нажимает кнопку «Сохранить». 6. Система проверяет корректность заполнения обязательных полей. 7. Система проверяет уникальность логина и адреса электронной почты. 8. Система сохраняет запись о новом тренере в базе данных. 9. Система отображает сообщение: «Тренер успешно добавлен».
Альтернативное направление развития варианта использования	Отмена добавления тренера. 1. Администратор нажимает кнопку «Отмена» до сохранения данных. 2. Система закрывает форму добавления без сохранения введенных данных. 3. Система возвращает администратора к списку тренеров.
Исключения	Ошибка валидации данных. 1. Система определяет, что одно или несколько обязательных полей не заполнены или заполнены неверно. 2. Система подсвечивает некорректно заполненные поля. 3. Система отображает сообщение: «Проверьте правильность заполнения полей».

Продолжение таблицы 2.2

Исключения	Логин или <i>email</i> уже существуют в системе. 1. Система обнаруживает совпадение логина или адреса электронной почты с уже существующей записью. 2. Система отображает сообщение: «Логин или <i>email</i> уже занят». 3. Администратор изменяет данные или отменяет добавление. Ошибка сохранения. 1. Система отображает сообщение об ошибке сохранения данных. 2. Запись о тренере не сохраняется.
------------	---

Таблица 2.3 – Описание варианта использования «Удаление тренера»

Идентификатор	UC-3 Удаление тренера	
Автор	Владислав	Дата создания: 17.03.2026
Основное действующее лицо	Администратор	
Описание	Администратор удаляет выбранного тренера из базы данных информационной системы тренажерного зала.	
Предварительные условия	<i>PRE-1.</i> Личность действующего администратора аутентифицирована. <i>PRE-2.</i> Администратор обладает правами удаления или деактивации тренеров. <i>PRE-3.</i> Удаляемый тренер существует в базе данных.	
Триггер	Администратор выбирает действие «Удалить тренера» в разделе управления тренерами.	
Выходные условия	<i>POST-1.</i> Тренер удален из базы данных или переведен в архивный статус. <i>POST-2.</i> Тренер больше не отображается среди активных тренеров. <i>POST-3.</i> Тренер недоступен для новых записей клиентов.	

Продолжение таблицы 2.3

Нормальное направление развития варианта использования	Удаление выбранного тренера. 1. Администратор открывает раздел управления тренерами. 2. Система отображает список тренеров. 3. Администратор выбирает тренера, которого необходимо удалить. 4. Администратор нажимает кнопку «Удалить». 5. Система отображает окно подтверждения удаления. 6. Администратор подтверждает удаление. 7. Система проверяет наличие активных или будущих записей клиентов у выбранного тренера. 8. Система обновляет список тренеров. 9. Система отображает сообщение: «Тренер успешно удален».
Альтернативное направление развития варианта использования	Отмена удаления тренера. 1. На этапе подтверждения удаления администратор нажимает кнопку «Отмена». 2. Система закрывает окно подтверждения. 3. Система возвращает администратора к списку тренеров без внесения изменений.
Исключения	Тренер связан с записями клиентов. 1. Система обнаруживает, что у тренера имеются активные или будущие записи клиентов. 2. Система переводит учетную запись в статус «Неактивен». 3. Система отображает сообщение: «Тренер деактивирован. Существующие записи клиентов сохранены». Ошибка удаления данных. 1. Система не может выполнить удаление из-за ошибки базы данных. 2. Система отображает сообщение: «Ошибка удаления тренера». 3. Запись о тренере не удаляется.

Таблица 2.4 – Описание варианта использования «Создание слота расписания»

Идентификатор	UC-4 Создание слота расписания	
Автор	Владислав	Дата создания: 17.03.2026
Основное действующее лицо	Тренер	
Описание	Тренер создает временной слот в собственном расписании, указывая дату, время и параметры тренировки для последующей записи клиентов.	
Триггер	Тренер указывает, что хочет добавить новый слот в свое расписание.	
Предварительные условия	<i>PRE-1.</i> Личность тренера аутентифицирована. <i>PRE-2.</i> Тренер имеет доступ к редактированию собственного расписания. <i>PRE-3.</i> База данных доступна для записи.	
Выходные условия	<i>POST-1.</i> Новый слот расписания сохраняется в базе данных. <i>POST-2.</i> Слот отображается в расписании тренера. <i>POST-3.</i> Слот становится доступным для записи клиентов.	
Нормальное направление развития варианта использования	Создание нового слота расписания. 1. Тренер открывает раздел управления расписанием. 2. Тренер выбирает действие «Создать слот расписания». 3. Система отображает форму создания нового слота. 4. Тренер вводит дату, время начала, продолжительность и количество участников. 5. Тренер нажимает кнопку «Сохранить». 6. Система проверяет корректность заполнения обязательных полей. 7. Система проверяет, не пересекается ли новый слот с уже существующими слотами тренера. 8. Система сохраняет новый слот расписания в базе данных. 9. Система отображает сообщение: «Слот расписания успешно создан».	

Продолжение таблицы 2.4

Альтернативное направление развития варианта использования	Отмена создания слота расписания. 1. Тренер нажимает кнопку «Отмена» до сохранения данных. 2. Система закрывает форму создания слота без сохранения введенных данных. 3. Система возвращает тренера к просмотру расписания.
Исключения	Конфликт времени с существующим слотом. 1. Система обнаруживает, что указанное время пересекается с уже существующим слотом расписания тренера. 2. Система отображает сообщение: «Слот пересекается с уже существующим». 3. Тренер изменяет дату или время слота либо отменяет создание. Ошибка заполнения данных. 1. Система определяет, что одно или несколько обязательных полей не заполнены или заполнены неверно. 2. Система отображает сообщение: «Проверьте правильность заполнения полей». Ошибка сохранения. 1. Система не может сохранить слот расписания в базе данных. 2. Система отображает сообщение: «Ошибка сохранения слота расписания». 3. Слот расписания не сохраняется.

2.4 Создание диаграмм взаимодействия

Диаграммы взаимодействия в *UML* предназначены для описания динамического поведения системы и отображения обмена сообщениями между ее компонентами в рамках конкретного сценария.

В процессе проектирования информационной системы тренажерного зала диаграммы взаимодействия позволяют проследить, каким образом клиент, мобильное приложение, серверная логика, база данных и приложение

тренера обмениваются данными при выполнении ключевого варианта использования – записи клиента на тренировку.

Существует два основных вида диаграмм взаимодействия: диаграмма последовательности и кооперативная диаграмма, которые различаются способом представления взаимодействия объектов системы. Диаграмма последовательности отображает сообщения между объектами в порядке их возникновения во времени. Кооперативная диаграмма показывает те же сообщения, но с акцентом на структурные связи между объектами, а не на временную последовательность [7].

Диаграмма последовательности для варианта использования «Запись на тренировку» представлена на рисунке 2.4. В диаграмме участвуют следующие объекты: клиент (актор), мобильное приложение (интерфейс клиента), менеджер записей (управляющий класс серверной логики), база данных и приложение тренера.

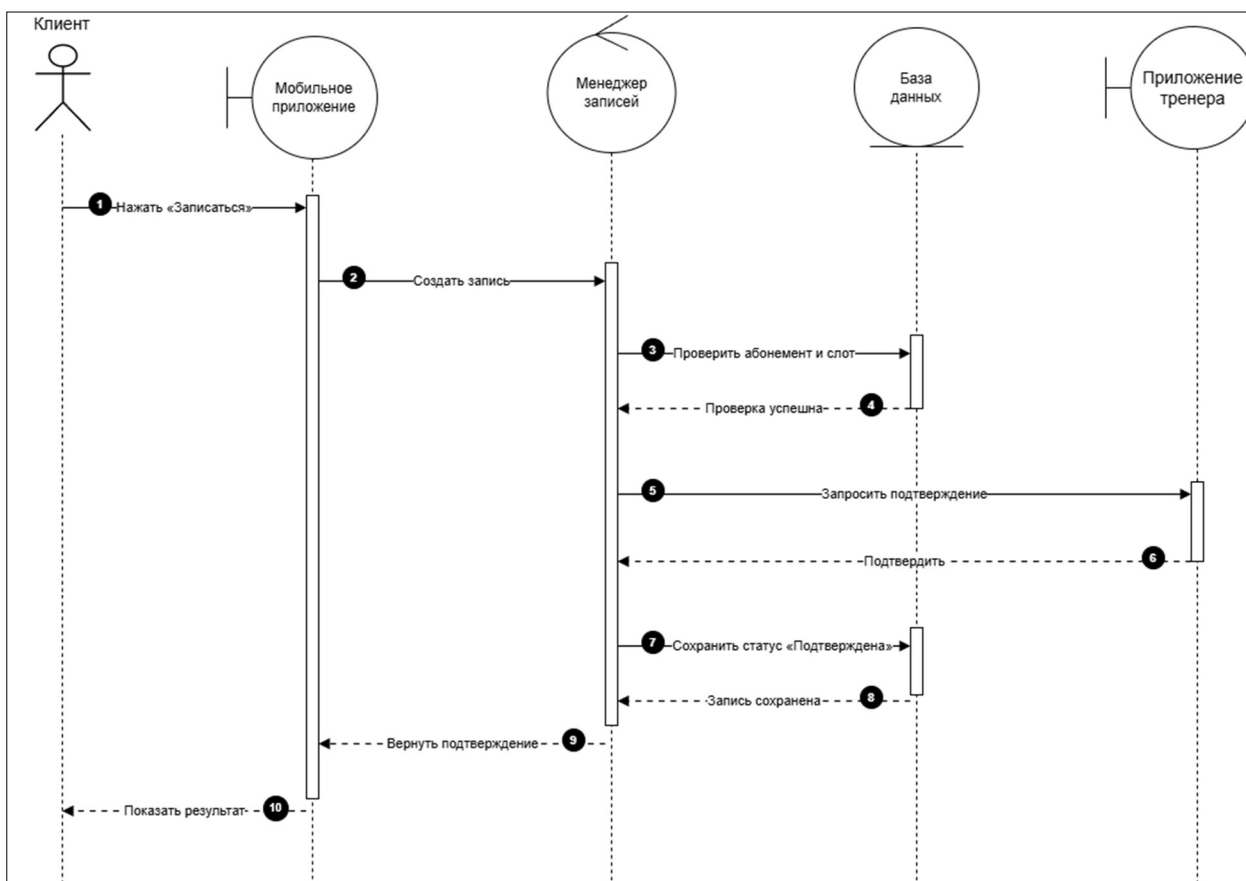


Рисунок 2.4 – Диаграмма последовательности варианта использования «Запись на тренировку»

Кооперативная диаграмма для того же сценария «Запись на тренировку» показывает структурные связи между объектами и нумерованные сообщения.

В отличие от диаграммы последовательности, временная ось здесь не отображается явно – вместо этого сообщения нумеруются и расставляются вдоль линий связей между объектами.

На кооперативной диаграмме видно, что мобильное приложение взаимодействует непосредственно с менеджером записей, который является центральным управляющим элементом процесса. Менеджер записей, в свою очередь, обращается к базе данных (для проверки и сохранения) и к приложению тренера (для подтверждения заявки). Такая архитектура соответствует принципу слабого связывания: клиентский интерфейс не взаимодействует напрямую ни с базой данных, ни с приложением тренера, делегируя всю логику управляющему классу [8].

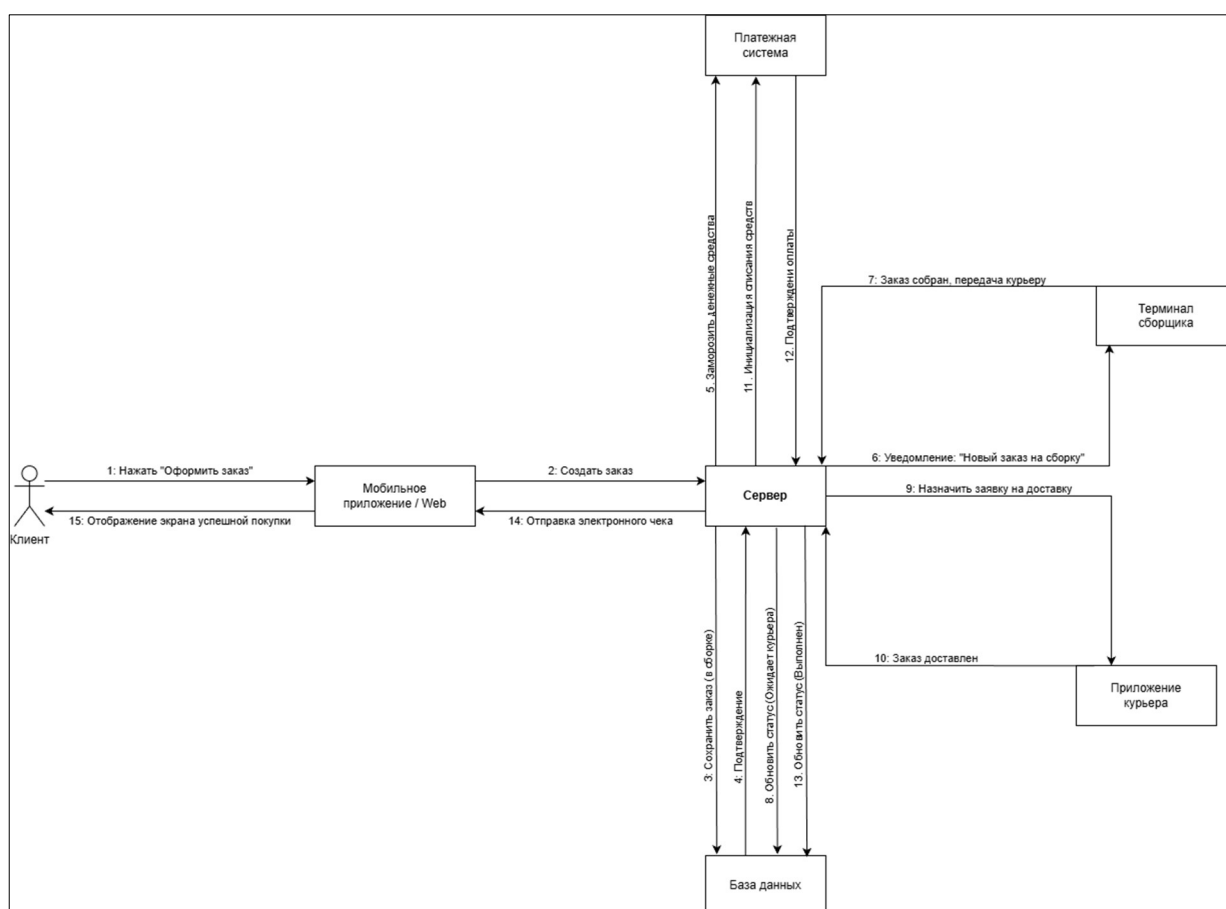


Рисунок 2.5 – Кооперативная диаграмма варианта использования «Запись на тренировку»

2.5 Создание диаграммы классов

Диаграмма классов является центральным инструментом статического моделирования в языке *UML*. Она отображает структуру системы сразу в трех измерениях: классы с их атрибутами и операциями, типы связей между

классами – ассоциация, агрегация, композиция, наследование, зависимость – и кратности этих связей. В отличие от диаграмм взаимодействия, описывающих поведение системы во времени, диаграмма классов фиксирует ее строение в статике. Построение диаграммы выполняется поэтапно: сначала выделяются классы, которые распределяются по трем стереотипам – граничные (*boundary*), отвечающие за взаимодействие с внешней средой; сущностные (*entity*), хранящие данные предметной области; и управляющие (*control*), инкапсулирующие бизнес-логику. Затем для каждого класса определяются атрибуты с указанием типов данных и операции с параметрами и возвращаемыми значениями, после чего устанавливаются связи с ролями и кратностями. Диаграмма классов является непосредственной основой для генерации скелетного кода системы.

Архитектура информационной системы тренажерного зала строится на иерархии пользователей. Базовым сущностным классом служит «Пользователь», объединяющий атрибуты, общие для всех категорий участников системы: уникальный идентификатор, ФИО, контактный телефон, адрес электронной почты, логин, хэш пароля и статус аккаунта. Посредством механизма наследования от «Пользователя» образованы три специализированных подкласса. Класс «Клиент» дополняется атрибутами даты регистрации и медицинских ограничений, а его операции обеспечивают просмотр расписания, запись на тренировку и ее отмену. Класс «Тренер» расширяется атрибутами специализации, стажа работы в годах и описания; его методы позволяют создавать тренировочные слоты, а также подтверждать и отклонять заявки клиентов. Класс «Администратор» наделен операциями добавления тренеров, изменения данных пользователей и обработки проблемных заявок.

Диаграмма классов представлена на рисунке 2.6.

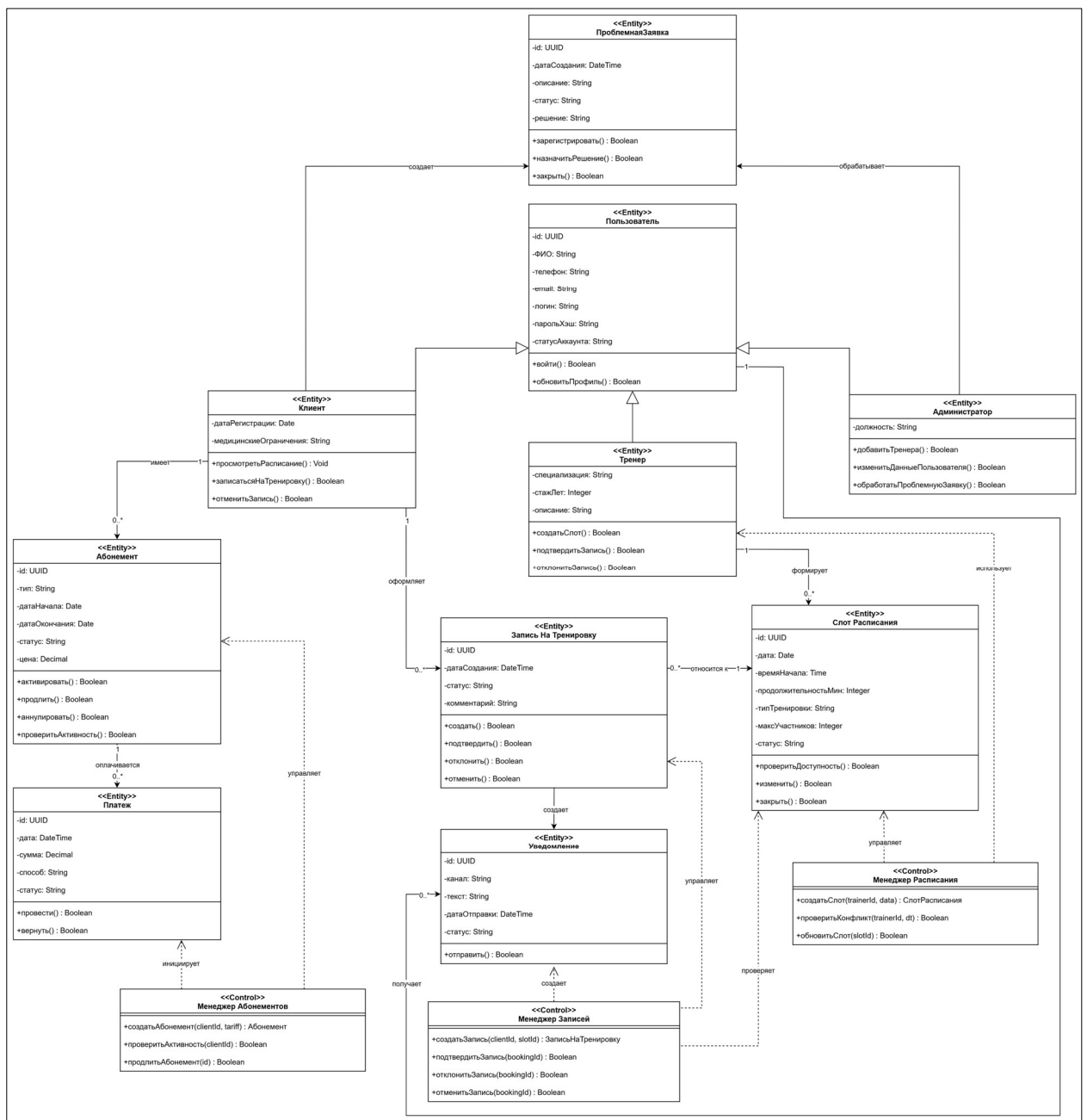


Рисунок 2.6 – Диаграмма классов

2.6 Создание диаграммы состояний

Диаграмма состояний в *UML* применяется для моделирования реактивного поведения объекта – описания того, как конкретная сущность системы реагирует на события, последовательно проходя через ряд качественно различных фаз. В отличие от диаграмм взаимодействия, фиксирующих обмен сообщениями между несколькими объектами, диаграмма состояний сосредоточена на жизненном цикле единственного объекта: от начального псевдосостояния до одного из терминальных.

Переходы между состояниями сопровождаются охраняющими условиями в квадратных скобках – логическими предикатами, разрешающими или запрещающими переход, – и действиями, выполняемыми в момент его совершения. Корректно построенная диаграмма состояний гарантирует, что реализованный на ее основе программный код никогда не переведет объект в логически недопустимое состояние, что особенно важно для бизнес-процессов, затрагивающих финансы пользователей или их запись к специалистам. Диаграммы состояний создаются не для всех классов, а лишь для тех объектов, жизненный цикл которых включает несколько принципиально различных фаз с разным поведением системы.

В информационной системе тренажерного зала диаграмма состояний разработана для класса-сущности «Запись на тренировку» – объекта, непосредственно затрагивающего интересы трех участников системы: клиента, тренера и администратора. Жизненный цикл начинается с начального псевдосостояния, из которого объект немедленно переходит в состояние «Создана». Здесь возникает первая точка ветвления: охраняющее условие [не активен] при неактивном абонементе клиента инициирует действие «отказ в записи» и завершает жизненный цикл в конечном псевдосостоянии. Если абонемент действителен, выполняется действие «отправить запрос тренеру» и объект переходит в состояние «Ожидает подтверждения тренера». Из этого состояния возможны три пути: тренер подтверждает заявку – выполняется действие «зафиксировать в расписании» и объект переходит в «Подтверждена»; тренер явно отклоняет запись – выполняется действие «уведомить клиента» и объект переходит в «Отклонена»; истекает отведенный срок ожидания ответа – выполняется действие «эскалация админу» и объект переходит в «Проблемная».

Из состояния «Проблемная» также предусмотрены два исхода: если администратор находит решение и подтверждает запись, объект переходит в «Подтверждена»; если урегулировать ситуацию не удастся, клиент уведомляется об отказе и объект переходит в «Отклонена». Из состояния «Подтверждена» жизненный цикл может завершиться двумя способами: клиент отменяет запись до начала тренировки, освобождая занятый слот расписания, – переход в «Отменена»; либо тренировка успешно проводится и посещение отмечается – переход в «Проведена». Состояния «Отменена», «Проведена» и «Отклонена» являются терминальными: каждое из них ведет непосредственно к конечному псевдосостоянию. Таким образом, диаграмма состояний полностью формализует все допустимые пути обработки заявки на тренировку и исключает возможность перехода объекта в логически некорректное состояние в процессе работы системы.

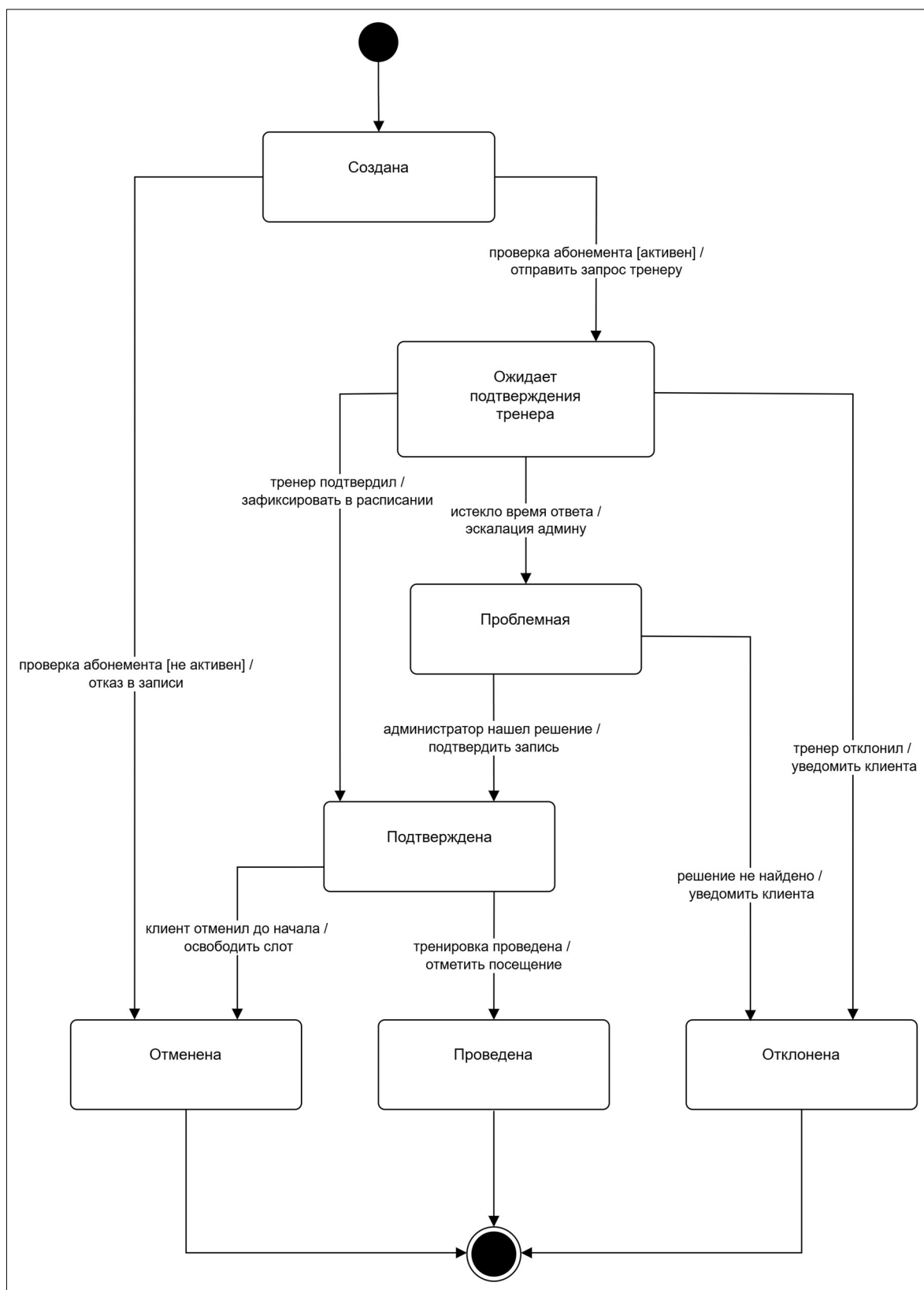


Рисунок 2.7 – Диаграмма состояний для класса-сущности
«Запись на тренировку»

2.7 Создание диаграммы активности

Среди поведенческих диаграмм *UML* диаграмма активности занимает особое место: в отличие от диаграмм последовательности, описывающих взаимодействие объектов в рамках одного временного потока, она ориентирована на целостный поток деятельности, охватывающий нескольких участников одновременно. По форме она близка к блок-схеме алгоритма, однако дополнена двумя существенными механизмами: форками и слияниями для моделирования параллельного выполнения задач и дорожками (*swimlanes*) для явного закрепления ответственности за каждым участником процесса. Дорожки превращают диаграмму из безликого потока блоков в структурированную схему с четким распределением ролей: читателю сразу видно, кто именно выполняет каждый шаг и в какой момент управление передается от одного участника к другому. Диаграмма активности особенно эффективна в сценариях с несколькими ветвлениями, где текстовое описание варианта использования становится громоздким: графическая форма представляет ту же информацию значительно компактнее и нагляднее.

Для информационной системы тренажерного зала диаграмма активности построена для бизнес-процесса «Запись клиента на тренировку» и разделена на три дорожки: «Клиент», «Информационная система» и «Тренер». Начальный узел расположен в дорожке «Клиент»: процесс инициируется выбором клиентом свободного слота в расписании тренировок. Управление немедленно переходит в дорожку «Информационная система», где выполняется проверка наличия активного абонемента. Узел принятия решения «Абонемент активен?» порождает две ветви: при отрицательном ответе система формирует отказ в записи, клиент получает соответствующее уведомление и процесс завершается. При положительном ответе система переходит к проверке доступности выбранного слота. Аналогичный узел «Слот свободен?» также допускает завершение процесса отказом. Если оба условия соблюдены, информационная система создает заявку со статусом «Ожидает подтверждения» и передает запрос в дорожку «Тренер». Тренер проверяет свое расписание: при подтверждении записи управление возвращается к информационной системе, которая фиксирует запись в расписании и уведомляет клиента об успехе; при отклонении заявки система обновляет статус на «Отклонена» и также уведомляет клиента. Таким образом, процесс завершается в одной из трех конечных точек: успешная запись, отказ до направления запроса тренеру или отказ по решению тренера.

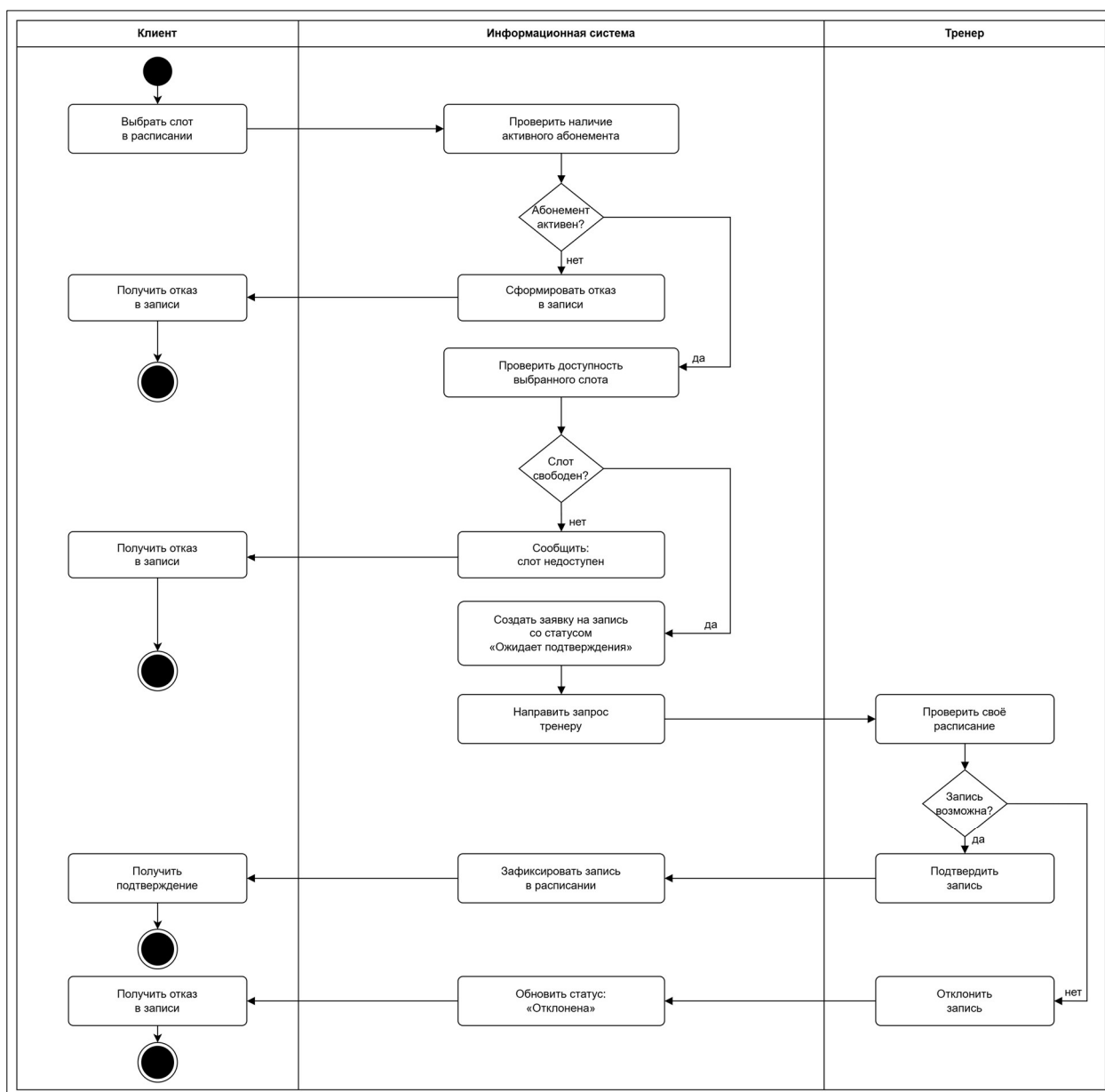


Рисунок 2.8 – Диаграмма активности процесса записи клиента на тренировку

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

3.1 Создание диаграммы компонентов

Диаграмма компонентов описывает программную систему с точки зрения ее физической реализации: из каких развертываемых модулей она состоит и какими зависимостями эти модули связаны. Если диаграмма классов раскрывает логическую структуру кода – иерархии объектов и их взаимосвязи, – то диаграмма компонентов показывает, как эти классы объединяются в самостоятельные исполняемые или библиотечные единицы, каждая из которых может разрабатываться, тестироваться и развертываться независимо. Компоненты взаимодействуют друг с другом через предоставляемые и требуемые интерфейсы, что обеспечивает четкое разграничение «контрактов» между модулями. Такой подход упрощает командную разработку: разработчик отдельного компонента знает только его интерфейс, не вникая во внутреннее устройство соседних модулей. Диаграмма компонентов строится на основе диаграммы классов и служит промежуточным шагом перед разработкой диаграммы развертывания.

Диаграмма компонентов информационной системы тренажерного зала отображает следующий набор программных модулей: мобильное приложение клиента (*iOS/Android*), реализующее интерфейс записи на тренировку, просмотра расписания и управления абонементом; личный кабинет тренера, обеспечивающий создание тренировочных слотов и обработку заявок клиентов; *web*-панель администратора, предоставляющая инструменты управления пользователями, абонементом и проблемными заявками. На серверной стороне выделены модуль управления записями и абонементом, реализующий ключевую бизнес-логику, а также компоненты интеграции с внешними сервисами – модуль взаимодействия с банковской системой для проведения платежных операций и модуль сервиса уведомлений для рассылки сообщений через *Email*, *SMS* и *Push*-каналы. Дополнительно представлен компонент работы с базой данных, инкапсулирующий все операции чтения и записи в СУБД *PostgreSQL*. Клиентские модули зависят от серверного модуля управления, который, в свою очередь, взаимодействует с компонентом базы данных и внешними интеграционными модулями.

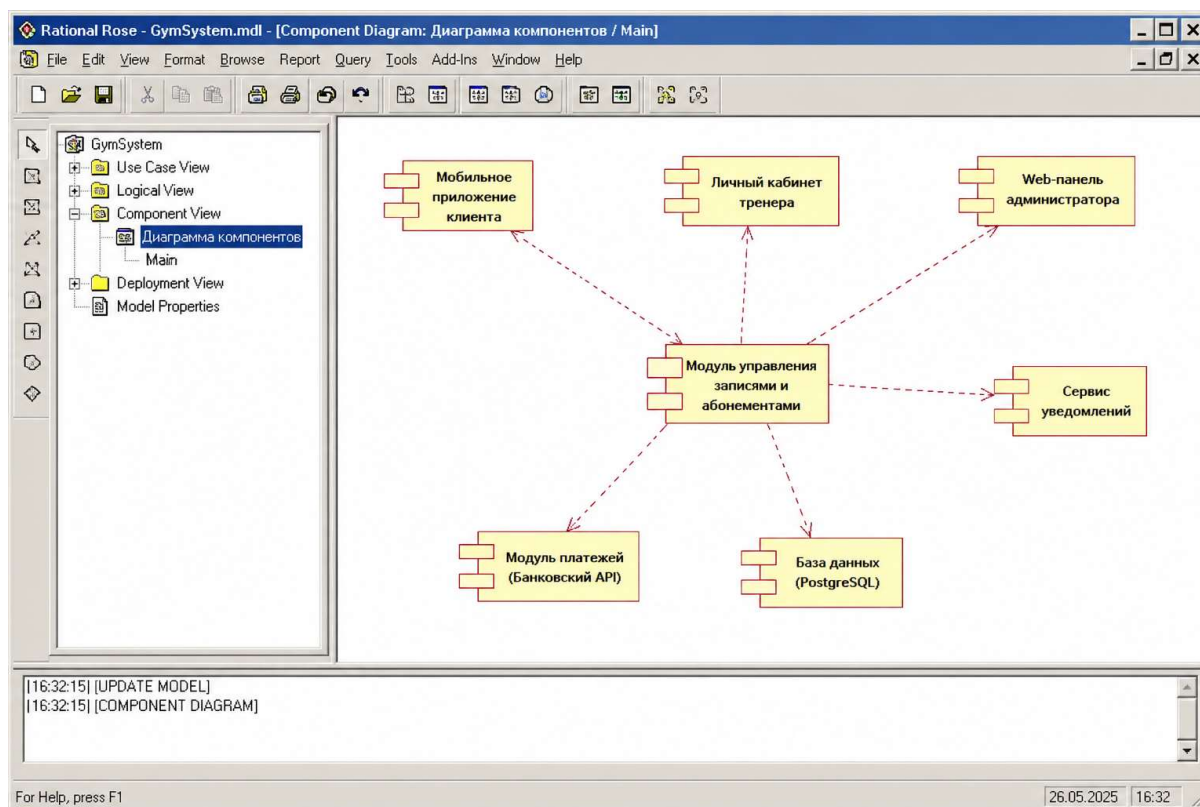


Рисунок 3.1 – Диаграмма компонентов системы тренажерного зала

3.2 Создание диаграммы развертывания

Диаграмма развертывания (*Deployment Diagram*) отображает физическую топологию системы: на каких вычислительных узлах (*nodes*) размещены программные артефакты и посредством каких коммуникационных путей они взаимодействуют. Каждый узел изображается в виде трехмерного прямоугольника со стереотипом – устройство (*device*), исполняющая среда или внешняя система, – а развернутые внутри него программные единицы отображаются как вложенные артефакты (*artifact*). Соединения между узлами аннотируются используемым протоколом передачи данных. Применение диаграммы развертывания позволяет обосновать требования к аппаратной инфраструктуре, явно указать точки интеграции с внешними системами и наглядно продемонстрировать, каким образом распределенные части системы взаимодействуют в условиях реальной эксплуатации.

Информационная система тренажерного зала реализует клиент-серверную архитектуру и включает несколько уровней развертывания. На клиентском уровне выделены три типа пользовательских устройств: смартфон клиента, на котором развернуто мобильное приложение для платформ *iOS* и *Android*; устройство тренера с личным кабинетом для управления расписанием, создания тренировочных слотов и подтверждения записей

клиентов; рабочая станция администратора с *web*-панелью для управления персоналом, абонементом и проблемными заявками. Все три клиентских приложения взаимодействуют с центральным сервером приложений, на котором развернут модуль управления записями и абонементом – основной обработчик запросов системы. Сервер приложений подключен к серверу базы данных, на котором функционирует СУБД *PostgreSQL*, обеспечивающая хранение всех сведений о пользователях, расписании, записях, абонементе и платежах. Помимо этого, сервер приложений взаимодействует с двумя внешними системами: банковской системой, через *API* которой проводятся и верифицируются платежные операции, и сервисом уведомлений, реализующим рассылку сообщений пользователям по каналам *Email*, *SMS* и *Push*.

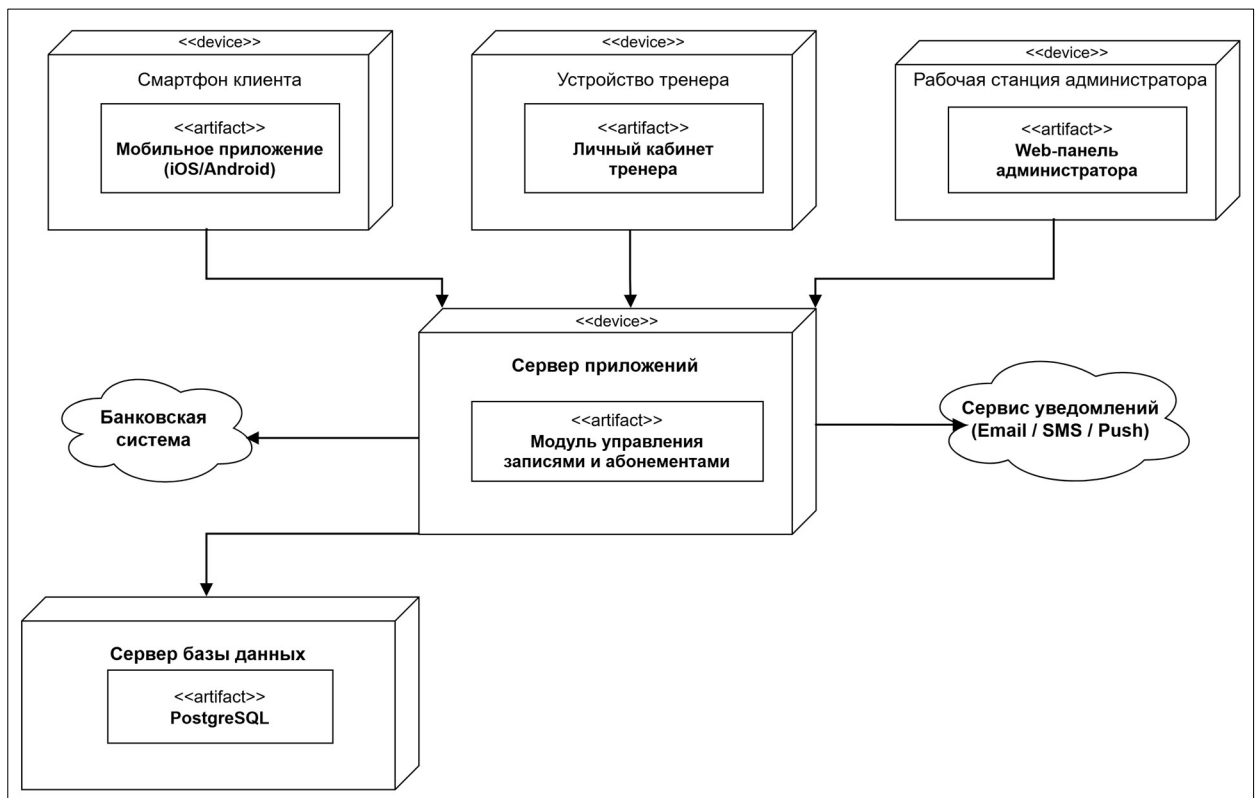


Рисунок 3.2 – Диаграмма развертывания

3.3 Генерация кода на C++

Средства автоматической генерации кода на языке C++, встроенные в CASE-средство *Rational Rose*, используют информацию, содержащуюся в диаграммах классов и компонентов, и формируют файлы заголовков и файлы реализации для выбранных классов. Создаваемый таким образом скелет

программы может быть уточнен путем прямого программирования на языке C++.

Всего генерируется два типа файлов: «.cpp» и «.h», что позволяет сразу приступить к дальнейшей разработке программной системы. Для выполнения генерации в спецификациях классов необходимо указать целевой язык *ANSI C++*, а также выбрать папку назначения, в которую будут сохранены создаваемые файлы. На основе заданной модели *Rational Rose* формирует заголовочные файлы с объявлениями атрибутов и методов, а также файлы реализации с заготовками тел методов.

В рамках курсового проекта была выполнена генерация скелетного кода для основных классов информационной системы тренажерного зала. Полученные файлы после генерации отображены на рисунке 3.3.

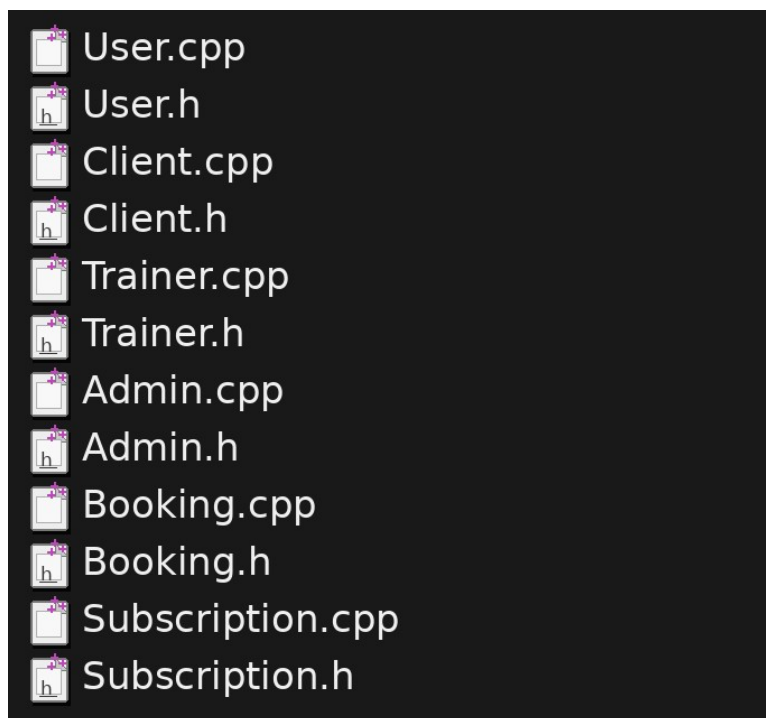


Рисунок 3.3 – Сгенерированные файлы

В результате были получены файлы с начальным программным кодом, который может использоваться как основа для дальнейшей реализации системы. Программист дополняет сгенерированные методы конкретной бизнес-логикой, сохраняя структуру классов, заданную на этапе объектно-ориентированного проектирования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была спроектирована информационная система тренажерного зала, предназначенная для автоматизации управления клиентами, тренерами, абонементом, расписанием, платежами и записями на тренировки.

В процессе проектирования проведен анализ предметной области, определены основные роли пользователей системы, сформулированы функциональные и нефункциональные требования, а также описаны ключевые бизнес-процессы тренажерного зала. Особое внимание уделено процессу записи клиента на тренировку, проверке активности абонента, контролю доступности тренировочного слота и обработке заявки тренером.

Для описания проектируемой системы построены диаграммы вариантов использования, классов, последовательности, кооперации, состояний и активности. Данные диаграммы позволяют отразить функциональные возможности системы, структуру основных сущностей, взаимодействие объектов и жизненный цикл записи на тренировку. Для основных вариантов использования составлены текстовые спецификации, раскрывающие нормальные, альтернативные и исключительные сценарии работы системы.

В разделе программной реализации с помощью CASE-средства *Rational Rose* созданы диаграмма компонентов и диаграмма развертывания, а также выполнена генерация фрагмента скелетного кода на языке C++. Полученный код представляет собой основу для дальнейшей программной реализации классов системы и наполнения методов конкретной бизнес-логикой.

Разработанная проектная модель включает необходимый набор UML-диаграмм, описание требований, спецификации вариантов использования и фрагменты программного кода. Полученные результаты могут быть использованы как основа для последующей разработки информационной системы, позволяющей клиентам самостоятельно записываться на тренировки, тренерам – управлять своим расписанием и обрабатывать заявки, а администраторам – контролировать клиентов, абонементы, платежи и работу тренажерного зала в единой системе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Фаулер, М. *UML. Основы* / М. Фаулер. – 3-е изд. – СПб. : Символ-Плюс, 2004. – 192 с.
- [2] Боггс, У. *UML и Rational Rose* / У. Боггс, М. Боггс. – М. : Лори, 2000. – 582 с.
- [3] Буч, Г. Язык *UML*. Руководство пользователя / Г. Буч, Д. Рамбо, А. Якобсон. – 2-е изд. – М. : ДМК Пресс, 2006. – 494 с.
- [4] Леоненков, А. В. Самоучитель *UML 2* / А. В. Леоненков. – СПб. : БХВ Петербург, 2007. – 576 с.
- [5] Якобсон, А. Унифицированный процесс разработки программного обеспечения / А. Якобсон, Г. Буч, Д. Рамбо. – СПб. : Питер, 2002. – 496 с.
- [6] Ларман, К. Применение *UML 2.0* и шаблонов проектирования / К. Ларман. – 3-е изд. – М. : Вильямс, 2007. – 736 с.
- [7] Вигерс, К. Разработка требований к программному обеспечению / К. Вигерс, Д. Битти. – 3-е изд. – СПб. : БХВ-Петербург, 2014. – 736 с.
- [8] Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – М. : Вильямс, 2003. – 1440 с.
- [9] Навроцкий, А. А. Дипломное проектирование по специальности «Автоматизированные системы обработки информации»: учеб.-метод. пособие / А. А. Навроцкий, Н. В. Батин. – Минск : БГУИР, 2018. – 66 с.
- [10] СТП 01-2024. Стандарт предприятия. Дипломные проекты (работы). Общие требования. – Минск: БГУИР, 2024 – 178 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код

Класс *User*

User.h

```
#ifndef USER_H_HEADER_INCLUDED_B6812A5
#define USER_H_HEADER_INCLUDED_B6812A5

//##ModelId=6812A50100E1
class User
{
public:
    //##ModelId=6812A5020001
    login();
    //##ModelId=6812A5020002
    updateProfile();

private:
    //##ModelId=6812A5030001
    string id;
    //##ModelId=6812A5030002
    string fio;
    //##ModelId=6812A5030003
    string email;
    //##ModelId=6812A5030004
    string userLogin;
    //##ModelId=6812A5030005
    string passwordHash;
    //##ModelId=6812A5030006
    string accountStatus;
};

#endif /* USER_H_HEADER_INCLUDED_B6812A5 */
```

User.cpp

```
#include "User.h"

//##ModelId=6812A5020001
User::login()
{
}

//##ModelId=6812A5020002
User::updateProfile()
{
}
```

Класс *Client*

Client.h

```
#ifndef CLIENT_H_HEADER_INCLUDED_B6812A6
#define CLIENT_H_HEADER_INCLUDED_B6812A6

#include "User.h"
```

```

//##ModelId=6812A60100E2
class Client : public User
{
public:
    //##ModelId=6812A6020001
    viewSchedule();
    //##ModelId=6812A6020002
    bookTraining();
    //##ModelId=6812A6020003
    cancelBooking();

private:
    //##ModelId=6812A6030001
    string registrationDate;
    //##ModelId=6812A6030002
    string medicalRestrictions;
};

#endif /* CLIENT_H_HEADER_INCLUDED_B6812A6 */

```

Client.cpp

```

#include "Client.h"

//##ModelId=6812A6020001
Client::viewSchedule()
{
}

//##ModelId=6812A6020002
Client::bookTraining()
{
}

//##ModelId=6812A6020003
Client::cancelBooking()
{
}

```

Класс *Trainer*

Trainer.h

```

#ifndef TRAINER_H_HEADER_INCLUDED_B6812A7
#define TRAINER_H_HEADER_INCLUDED_B6812A7

#include "User.h"

//##ModelId=6812A70100E3
class Trainer : public User
{
public:
    //##ModelId=6812A7020001
    createSlot();
    //##ModelId=6812A7020002
    confirmBooking();
    //##ModelId=6812A7020003
    rejectBooking();
}

```

```

private:
    //##ModelId=6812A7030001
    string specialization;
    //##ModelId=6812A7030002
    int experience;
    //##ModelId=6812A7030003
    string description;
};
#endif /* TRAINER_H_HEADER_INCLUDED_B6812A7 */

```

Trainer.cpp

```

#include "Trainer.h"
//##ModelId=6812A7020001
Trainer::createSlot()
{
}
//##ModelId=6812A7020002
Trainer::confirmBooking()
{
}
//##ModelId=6812A7020003
Trainer::rejectBooking()
{
}

```

Класс Admin

Admin.h

```

#ifndef ADMIN_H_HEADER_INCLUDED_B6812A8
#define ADMIN_H_HEADER_INCLUDED_B6812A8
#include "User.h"
//##ModelId=6812A80100E4
class Admin : public User
{
public:
    //##ModelId=6812A8020001
    addTrainer();
    //##ModelId=6812A8020002
    editUserData();
    //##ModelId=6812A8020003
    processProblemRequest();
private:
    //##ModelId=6812A8030001
    string position;
};
#endif /* ADMIN_H_HEADER_INCLUDED_B6812A8 */

```

Admin.cpp

```

#include "Admin.h"
//##ModelId=6812A8020001
Admin::addTrainer()
{
}
//##ModelId=6812A8020002
Admin::editUserData()
{
}

```

```

}
//##ModelId=6812A8020003
Admin::processProblemRequest()
{
}

```

Класс *Booking*

Booking.h

```

#ifndef BOOKING_H_HEADER_INCLUDED_B6812AB
#define BOOKING_H_HEADER_INCLUDED_B6812AB
//##ModelId=6812AB0100E7
class Booking
{
public:
    //##ModelId=6812AB020001
    create();
    //##ModelId=6812AB020002
    confirm();
    //##ModelId=6812AB020003
    reject();
    //##ModelId=6812AB020004
    cancel();
private:
    //##ModelId=6812AB030001
    string id;
    //##ModelId=6812AB030002
    string createdAt;
    //##ModelId=6812AB030003
    string status;
    //##ModelId=6812AB030004
    string comment;
};
#endif /* BOOKING_H_HEADER_INCLUDED_B6812AB */

```

Booking.cpp

```

#include "Booking.h"
//##ModelId=6812AB020001
Booking::create()
{
}
//##ModelId=6812AB020002
Booking::confirm()
{
}
//##ModelId=6812AB020003
Booking::reject()
{
}
//##ModelId=6812AB020004
Booking::cancel()
{
}

```

Класс *Subscription*

Subscription.h

```
#ifndef SUBSCRIPTION_H_HEADER_INCLUDED_B6812A9
#define SUBSCRIPTION_H_HEADER_INCLUDED_B6812A9
//##ModelId=6812A90100E5
class Subscription
{
public:
    //##ModelId=6812A9020001
    activate();
    //##ModelId=6812A9020002
    extend();
    //##ModelId=6812A9020003
    cancel();
    //##ModelId=6812A9020004
    checkActivity();

private:
    //##ModelId=6812A9030001
    string id;
    //##ModelId=6812A9030002
    string type;
    //##ModelId=6812A9030003
    string startDate;
    //##ModelId=6812A9030004
    string endDate;
    //##ModelId=6812A9030005
    string status;
    //##ModelId=6812A9030006
    double price;
};
#endif /* SUBSCRIPTION_H_HEADER_INCLUDED_B6812A9 */
```

Subscription.cpp

```
#include "Subscription.h"
//##ModelId=6812A9020001
Subscription::activate()
{
}
//##ModelId=6812A9020002
Subscription::extend()
{
}
//##ModelId=6812A9020003
Subscription::cancel()
{
}
//##ModelId=6812A9020004
Subscription::checkActivity()
{
}
```

[illegible]